Aconcagua CFD

User Guide

Workflow to calculate the drag of a finned rocket with OpenFOAM: geometry, mesh, cases, sweeps, measurement and validation

Document	ACG-CFD-UG-001
Revision	B
Date	2026-10-18
Repository	`github.com/vnemegabriel/CFD-ITBA-ROCKETRY`, directory `flightModel/`
Organization	ITBA Rocketry

Document control

Revision history

Rev.	Date	Change
A	2026-10-18	First issue. Replaced the Markdown files of `flightModel/docs/`.
B	2026-10-18	The guide gives only the method. Measured results moved to the technical report ACG-CFD-TR-001. Parameter values are not copied: each parameter has its function, the principle that sets it, and its file. Content specific to one computer removed. Diagrams with fixed geometric meaning.

Maintenance of this document

- This guide is the only user documentation of `flightModel/`. The dictionaries and scripts are the only source of the parameter values: this guide tells where each value is, and why, but does not copy it.
- When a change to a script changes a procedure, change the guide in the same commit. Compile it with `make` (Section 3.5) and commit the PDF.
- Results go to the technical report, not to this guide.

Contents

Document control i

About this guide xi

1 Overview 1
- 1.1 Purpose of the workflow . . . 1
- 1.2 Workflow . . . 1
- 1.3 Two solvers . . . 2
- 1.4 Directory structure . . . 3

2 Concepts you need 5
- 2.1 An OpenFOAM case . . . 5
- 2.2 Finite volumes and the mesh . . . 5
- 2.3 Compressible flow regimes . . . 5
- 2.4 Turbulence, RANS and y^+ . . . 6
- 2.5 Steady solution by pseudo-time marching . . . 6
- 2.6 Force coefficients . . . 6
- 2.7 Learning path . . . 6

3 Installation and setup 9
- 3.1 Requirements . . . 9
- 3.2 Choose an installation . . . 9
  - 3.2.1 Container . . . 9
  - 3.2.2 Native installation on Ubuntu, Debian or WSL 2 . . . 10
- 3.3 First run . . . 11
- 3.4 Prepare the computer for a study . . . 11
- 3.5 Build this guide . . . 11

4 Geometry 13
- 4.1 Requirements of the surface . . . 13
- 4.2 Design rules of the surface . . . 13
- 4.3 Write the geometry . . . 14
- 4.4 Use a new geometry . . . 14

5 Mesh 17
- 5.1 Octree principle . . . 17
- 5.2 Refinement regions . . . 17
- 5.3 Boundary layers . . . 18
- 5.4 Domain . . . 19
- 5.5 Half mesh . . . 20

5.6 Patches . . . 20
5.7 meshInfo . . . 20
5.8 Make a mesh . . . 21
5.9 Examine a mesh . . . 21

6 Cases **23**
6.1 Make a case . . . 23
6.1.1 Command . . . 23
6.1.2 Steps that newCase.sh does . . . 24
6.2 Run a case . . . 24
6.2.1 Command . . . 24
6.2.2 Steps that Allrun does . . . 25
6.2.3 Process binding . . . 26
6.2.4 End of a run . . . 26
6.2.5 Continue a run . . . 26
6.2.6 Start a run again from zero . . . 27
6.3 Start from another run . . . 27
6.4 Refine the mesh near the fins (optional) . . . 28

7 Studies and sweeps **29**
7.1 Design of a study . . . 29
7.1.1 Chains of runs . . . 29
7.1.2 Mesh series for the convergence study . . . 29
7.2 Make the meshes . . . 30
7.3 Run the cases . . . 30
7.3.1 Start a sweep . . . 31
7.3.2 Stop a sweep . . . 31
7.4 Flight conditions . . . 31
7.5 Rules . . . 31

8 Solvers and models **33**
8.1 Two solvers . . . 33
8.2 Files of the templates . . . 33
8.3 Flight condition . . . 34
8.4 Numerical method of rhoCentralFoam . . . 34
8.5 Local time step, Courant ramp and stop criterion . . . 35
8.5.1 Local time step . . . 35
8.5.2 Courant ramp (central template) . . . 35
8.5.3 config.json . . . 36
8.5.4 Stop criterion . . . 36
8.6 Turbulence . . . 36
8.6.1 Wall treatment . . . 37
8.7 Gas . . . 37
8.8 Boundary conditions . . . 37
8.9 Function objects . . . 38
8.10 Discretization schemes and linear solvers . . . 38
8.11 rhoPimpleFoam template . . . 39
8.12 Start of the supersonic cases . . . 39

9 Results and measurement **41**

9.1 Outputs of a run . . . 41
9.2 run.info . . . 42
9.3 Measurement of the computer . . . 43
9.3.1 Requirements . . . 44
9.3.2 Procedure to get a comparable cost . . . 44
9.3.3 Do not divide by the clock frequency . . . 44
9.3.4 Runs without this measurement . . . 45
9.4 Monitor a run . . . 45
9.5 Table of a sweep . . . 45
9.6 Disk . . . 45
9.7 Notification . . . 46

10 Comparison with Nektar++ **47**
10.1 What must be the same . . . 47
10.2 What to compare . . . 48
10.3 How to measure in the same way . . . 48

11 Flight data **49**
11.1 Command . . . 49
11.2 Method . . . 49
11.3 Angle of attack of the flight . . . 51
11.4 Sources of uncertainty . . . 51

12 Validation **53**
12.1 Checks on each run . . . 53
12.2 Subsonic flow: Barrowman . . . 53
12.3 Transonic and supersonic flow . . . 54
12.4 Mesh convergence . . . 54
12.5 Record . . . 55

13 Troubleshooting **57**
13.1 mapFields on the same mesh . . . 58
13.2 The Courant number with a local time step . . . 58
13.3 Decimal comma . . . 58
13.4 Rules for a change to the scripts . . . 58

A Quick reference **59**
A.1 Commands . . . 59
A.2 Run names . . . 59
A.3 Where each value is . . . 60

B Glossary **61**
B.1 Terms and abbreviations . . . 61
B.2 Symbols . . . 61

List of Figures

1.1 Workflow. Each frame is a stage; left to right is the sequence. 1
1.2 Decisions that depend on the free-stream Mach number. 2

5.1 Nesting of the refinement regions, upstream to downstream. 17
5.2 Steps of `Allmesh`: surface operations, then volume operations. 22

6.1 How `newCase.sh` assembles a case from the templates and a mesh. 24
6.2 Steps of `Allrun`. 25

7.1 Chains of runs. Left to right: from coarse to fine. Each arrow is an `--init`. . . 30

8.1 Phases of a run of the central template along the iterations. The diamonds are the keys of `config.json` that place each switch. 35

9.1 Measurement during the solver and its record in `run.info`. 43

11.1 Calculation of the flight drag coefficient, from the telemetry (top) to the result (bottom). 50

List of Tables

1 Geometric conventions of the diagrams. xii

1.1 Contents of `flightModel/`. 3

2.1 OpenFOAM tutorials to do before this case, in this sequence. Paths are relative to `$FOAM_TUTORIALS`. 7

3.1 Requirements. 9

4.1 Requirements of the STL file. 13
4.2 Design rules of the surface. 14

5.1 Function of each refinement and the principle that sets its level. 18
5.2 Entries of `meshInfo`. 20

6.1 Options of `newCase.sh`. 23
6.2 Flow angles that each mesh sector permits. 26

8.1 Files of the templates and their function. 33

9.1 Outputs of a run. `<t0>` is the start time: 0 for a new run. 41
9.2 Samples for the comparison with another solver (function object `samples`). . . 42
9.3 Entries of `run.info`. 42
9.4 Block `measured` of `run.info`. 43

10.1 Items that the Nektar++ run must reproduce. 47

12.1 Checks on each run. 53

13.1 Symptoms, causes and remedies. 57

About this guide

Purpose

This guide tells you how to calculate the drag coefficient of a finned rocket as a function of the Mach number and the angle of attack with OpenFOAM, how to estimate the discretization error, and how to compare the result with flight data and with another solver. For each step it gives the procedure and the principle that justifies it.

Audience

Engineering students and engineers who replicate or extend the workflow, on any computer. You need a basic course of fluid mechanics and the use of a command line. Chapter 2 gives the CFD concepts that the workflow uses and a learning path for a reader with little CFD practice.

Scope

The directory `flightModel/`: geometry, mesh, case templates, sweeps, post-processing, measurement of the computer and flight data. The guide applies to any computer: with the container on Windows, macOS or Linux, or with a native installation on Linux (Section 3.2).

How to use this guide

If you want to	Read
Understand the workflow	Chapter 1
Learn the CFD concepts first	Chapter 2
Install and make the first run	Sections 3.2 and 3.3
Run a study	Chapter 7
Change the geometry or the mesh	Chapters 4 and 5
Know why the solver is set up as it is	Chapter 8
Measure the cost of a run	Chapter 9
Compare with another solver	Chapter 10
Compare with the flight	Chapter 11
Validate a result	Chapter 12
Correct a problem	Chapter 13

Conventions

Convention	Meaning
`mesh/Allmesh`	A file, a directory, a command, a dictionary entry or a value.
`<mesh-dir>`, `[stl-file]`	A value that you must supply; an optional argument.
Paths without a leading `/`	Relative to `flightModel/`.
$h_0, h_1, \ldots$	The meshes of a study, from coarse to fine. h_0 only starts the runs.

Procedures are numbered steps, one action in each step. Table 1 gives the meaning of the geometry of the diagrams.

Table 1: Geometric conventions of the diagrams.

Geometry	Meaning
Left to right	The workflow, or the time, or the refinement from coarse to fine.
Top to bottom	From the input to the result, inside one stage.
A box inside a box	A region of a finer mesh level inside a coarser one.
A diamond	A decision.
A cylinder	Data on disk.
A frame with a title	One stage or one script.

Notices

CAUTION

A CAUTION tells you about a condition that can damage data, give an incorrect result without a message, or waste computer time.

NOTE

A NOTE gives information that helps you to do a step or to understand a result.

Writing standard

The structure follows ISO/IEC/IEEE 26514 [1]: concepts, procedures and reference information are separate. The text follows the rules of ASD-STE100 Simplified Technical English [2]. Technical names stay as the OpenFOAM documentation writes them.

Related documents

Document	Contents
ACG-CFD-TR-001, `docs/report/`	Technical report: the results of the workflow.
`CLAUDE.md`	Rules for the people and the agents who change the repository.
`case-central/README.md`, `case-pimple/README.md`	Short description of each template.
OpenFOAM v2412 user guide	`https://www.openfoam.com/documentation/user-guide`

Chapter 1

Overview

1.1 Purpose of the workflow

The workflow calculates the drag coefficient $C_D(M, \alpha)$ of a finned rocket at the flight conditions of a real flight. It runs each condition on a series of meshes to estimate the discretization error, and it records the cost of each run so that another solver can be compared on the same cases.

The mesh must resolve three flow features. Their order is the order of their contribution to the drag error:

1. **Shock waves** at transonic and supersonic speeds. The wave drag depends on the position and the strength of the shocks.
2. **The base wake.** The base pressure sets the base drag, a large part of the total drag of a rocket with a blunt base.
3. **The wall shear stress.** It sets the friction drag. It depends on the resolution of the boundary layer, that is, on y^+.

Flow features that do not change the axial force, such as the tip vortex of the fins, do not get refinement.

1.2 Workflow

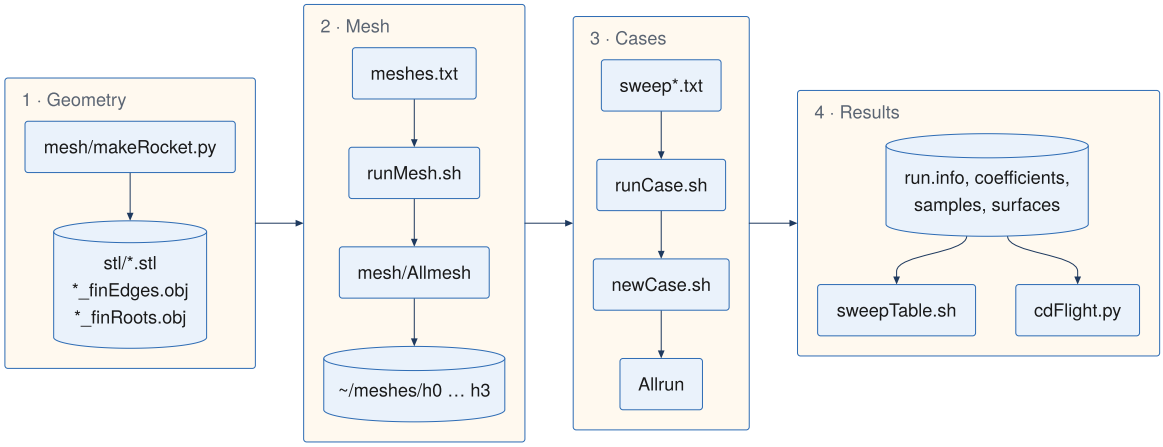


Figure 1.1: Workflow. Each frame is a stage; left to right is the sequence.

1. **Geometry.** `mesh/makeRocket.py` writes the surface from the dimensions of the rocket (Chapter 4).

2. **Mesh.** `mesh/Allmesh` makes one octree mesh; `runMesh.sh` makes the series of meshes of a study (Chapter 5).

3. **Cases.** `newCase.sh` makes a case from a template, a mesh and a flight condition; `Allrun` runs it; `runCase.sh` does this for each line of a sweep file (Chapters 6 and 7).

4. **Results.** Each run records its coefficients, its samples and its cost (Chapter 9). The post-processing compares the runs with the flight (Chapter 11) and with another solver (Chapter 10).

1.3 Two solvers

Figure 1.2 shows how the flow regime selects the solver, the outlet pressure condition and the initial field. Chapter 8 gives the reasons.

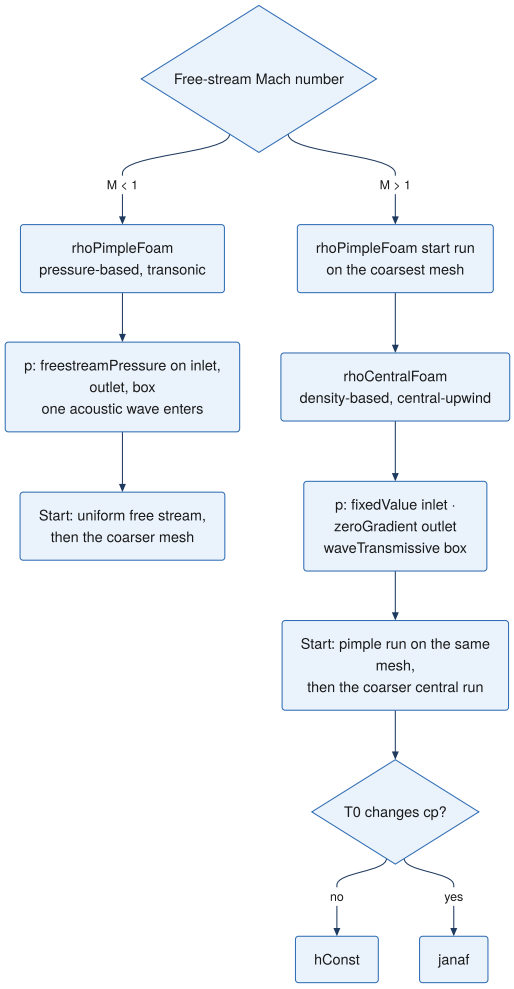


Figure 1.2: Decisions that depend on the free-stream Mach number.

1.4 Directory structure

Table 1.1: Contents of `flightModel/`.

Path	Contents
`mesh/makeRocket.py`	Writes the geometry: the STL file and its edge files.
`mesh/stl/`	The geometry files.
`mesh/Allmesh`, `mesh/snapPlane.py`	Make one mesh; flatten the cut of a half mesh.
`mesh/system/`	`meshDict` and the other dictionaries of the mesh.
`mesh/meshInfo`	Reference values and geometry data that the cases read.
`case-central/`, `case-pimple/`	Case templates. `case-pimple/` holds only the files that differ.
`common/`	`Allrun`, `Allrefine`, `machineLog.py`: copied into each case.
`newCase.sh`	Makes one case.
`runMesh.sh`, `meshes.txt`	Make the meshes of a study.
`runCase.sh`, `sweep*.txt`	Make and run the cases of a sweep.
`post/`	`monitor.sh`, `sweepTable.sh`, `firstLayer.py`.
`flightData/`	Flight telemetry and `cdFlight.py`.
`docs/guide/`, `docs/report/`	This guide; the technical report.

Chapter 2

Concepts you need

This chapter is for a reader with little CFD practice. It gives each concept that the workflow uses, why the workflow needs it, and where to learn more. Do the OpenFOAM tutorials of Table 2.1 before your first run of this case.

2.1 An OpenFOAM case

An OpenFOAM case is a directory with three parts:

Directory	Contents
`0/`	The initial and boundary conditions of each field (p, $\mathbf{U}$, T, ...). The templates keep them in `0.orig/` and copy them to `0/` at the start.
`constant/`	The mesh (`polyMesh/`) and the physical models (gas, turbulence).
`system/`	The numerical settings: time control (`controlDict`), discretization (`fvSchemes`), linear solvers (`fvSolution`), parallel decomposition.

Each file is a dictionary of `keyword value;` entries. `foamDictionary` reads and writes them from the shell. The OpenFOAM user guide explains the structure.

2.2 Finite volumes and the mesh

The finite-volume method divides the domain into cells and solves the conservation of mass, momentum and energy for each cell: what enters through the faces equals what leaves plus what accumulates [3, 4]. The value at a face comes from the values at the cell centres with an interpolation scheme. The accuracy thus depends on the size and the shape of the cells where the flow changes fast: shocks, the boundary layer, the wake. A refined mesh reduces the discretization error; the mesh-convergence study of Section 12.4 measures it.

Mesh quality has three common measures. *Non-orthogonality* is the angle between a face normal and the line between the two cell centres. *Skewness* is the distance of that line from the face centre. A *negative volume* is an inverted cell, which makes the mesh unusable.

2.3 Compressible flow regimes

The Mach number $M = U/a$ sets the physics and the numerics [5]:

Regime	What happens
$M < 0.3$	Density changes are small.
$0.3 < M < 0.8$	Compressible, without shocks.
$0.8 < M < 1.2$	Transonic: local shocks, a fast rise of the drag.
$M > 1.2$	Supersonic: a bow shock at the nose, oblique shocks at the fins, an expansion and a wake at the base. Information travels only downstream, inside the Mach cone.

This is why the workflow uses two solvers (Section 8.1) and different outlet conditions (Section 8.8).

2.4 Turbulence, RANS and y^+

The flow around a rocket is turbulent. A RANS model solves for the mean flow and models the effect of the turbulent fluctuations with an eddy viscosity ν_t [6, 7]. The k–ω SST model of this workflow solves two equations, for the turbulent kinetic energy k and its specific dissipation rate ω.

Near a wall the velocity changes over a very small distance. $y^+ = y\,u_\tau/\nu$ is the distance of the first cell centre from the wall in wall units. Below $y^+ \approx 5$ the cell is in the viscous sublayer; above $y^+ \approx 30$, in the log layer; between them, in the buffer layer. A wall function gives the wall shear stress from the first cell when the mesh does not resolve the sublayer (Section 8.6.1).

2.5 Steady solution by pseudo-time marching

The workflow wants the steady flow. The solvers march in a pseudo-time from an initial field until the solution stops changing (Section 8.5.1). Each iteration reduces the *residuals*, the error of the discrete equations, and the force coefficients become flat. The run stops when the mean of C_D over a window is constant (Section 8.5.4). The *Courant number* $Co = U\,\Delta t/\Delta x$ limits the step of each cell: a large step makes the march unstable.

2.6 Force coefficients

The force on the rocket, divided by the dynamic pressure $\frac{1}{2}\rho_\infty U_\infty^2$ and a reference area, gives the coefficients. C_D (drag) and C_L (lift) are along and normal to the free stream; C_A (axial) and C_N (normal) are along and normal to the body axis. C_m is the moment coefficient about a reference point. The centre of pressure is where the normal force acts; a rocket is stable if it is behind the centre of gravity.

2.7 Learning path

To copy and run a tutorial:

```
cp -r $FOAM_TUTORIALS/incompressible/icoFoam/cavity/cavity □/cavity
cd □/cavity && ./Allrun   # or: blockMesh && icoFoam
```

Table 2.1: OpenFOAM tutorials to do before this case, in this sequence. Paths are relative to `$FOAM_TUTORIALS`.

Tutorial	What it teaches
`incompressible/icoFoam/cavity/cavity`	The case structure, a run, ParaView.
`incompressible/simpleFoam/pitzDaily`	A RANS model, wall functions, convergence of a steady solution.
`compressible/rhoPimpleFoam/RAS/aerofoilNACA0012`	The pressure-based compressible solver with RANS on an external flow.
`compressible/rhoCentralFoam/forwardStep`	The density-based solver and a supersonic flow with shocks.
`compressible/rhoCentralFoam/wedge15Ma5`	An oblique shock with an exact solution (Section 12.3).

For the theory, read the finite-volume method in Versteeg and Malalasekera [3] or, with OpenFOAM examples, Moukalled et al. [4]; compressible flow in Anderson [5]; turbulence modelling in Wilcox [6].

Chapter 3

Installation and setup

3.1 Requirements

Table 3.1: Requirements.

Item	Requirement
OpenFOAM	ESI v2412 with cfMesh (`cartesianMesh`), and its Open MPI.
Tools	`bash`, `rsync`, `awk`, GNU `sed`, `xz`, `/usr/bin/time`, `jinja2-cli`, `git`.
Python 3	Standard library; `matplotlib` for `cdFlight.py`.
`perf`	Optional: counts the CPU cycles and the energy of each run. Native Linux only.
Memory	Proportional to the number of cells. Measure `maxRSSkB` of a run on the coarsest mesh (Section 9.2) and scale it with the number of cells before you run a finer mesh. `cartesianMesh` needs the memory of the full rocket also for a half mesh.

3.2 Choose an installation

Computer	Installation	Note
Any (Windows, macOS, Linux)	Container (Section 3.2.1)	The same environment on all computers. Recommended for a first replication.
Ubuntu or Debian	Native (Section 3.2.2)	Best performance; needed for the measurement with `perf`.
Windows	Native in WSL 2 (Section 3.2.2)	Ubuntu in WSL 2, then the native procedure.
macOS	Container	No native OpenFOAM package. On Apple silicon the container runs emulated, slower.

3.2.1 Container

The repository has a container recipe, `container/Dockerfile`, with OpenFOAM, cfMesh, Open MPI and all the tools. The meshes and the runs go to a volume, so they stay when the container stops.

1. Install Docker (Docker Desktop on Windows and macOS).
2. Get the repository:

```
git clone https://github.com/vnemegabriel/CFD-ITBA-ROCKETRY.git
cd CFD-ITBA-ROCKETRY
```

3. Build the image once:

```
docker build -t aconcagua-cfd -f container/Dockerfile .
```

   On Apple silicon, add `--platform linux/amd64`.

4. Start a shell in the container:

```
docker run --rm -it -v "$PWD":/work -v aconcagua-data:/data aconcagua-cfd
```

   The shell starts in `/work/flightModel` with OpenFOAM loaded; `~/meshes` and `~/runs` are in the volume `aconcagua-data`.

5. Do the first run (Section 3.3).

NOTE

Apptainer (Singularity) can run the same image on a computing cluster without root: `apptainer build aconcagua.sif docker-daemon://aconcagua-cfd:latest`.

3.2.2 Native installation on Ubuntu, Debian or WSL 2

1. On Windows only: install WSL 2 with Ubuntu (`wsl --install -d Ubuntu`), and do the next steps in the Ubuntu terminal.
2. Add the OpenFOAM repository of ESI and install OpenFOAM with its modules:

```
curl -s https://dl.openfoam.com/add-debian-repo.sh | sudo bash
sudo apt-get install openfoam2412-default
```

3. Install the tools:

```
sudo apt-get install rsync xz-utils gawk time git python3-matplotlib pipx
pipx install jinja2-cli
```

4. Get the repository (`git clone`, as in Section 3.2.1).
5. Load OpenFOAM in each new shell, or add the line to `~/.bashrc`:

```
source /usr/lib/openfoam/openfoam2412/etc/bashrc
```

   If OpenFOAM is in another place, use the `etc/bashrc` of that installation. All scripts stop with `source the OpenFOAM bashrc first` if this step is missing.

CAUTION

On WSL, put the repository, the meshes and the runs on the Linux file system (`~/...`), not in `/mnt/c`: the input and output of OpenFOAM become several times slower there.

3.3 First run

The first run checks the installation on the coarsest mesh of the study, h_0. h_0 always starts the chains of runs (Section 7.1), so this run is also the first run of a real study.

1. Make the mesh (Section 5.8):

```
cd flightModel
mesh/Allmesh □/meshes/h0 $(grep '^h0 ' meshes.txt | cut -d' ' -f2-)
```

2. Make and run one case at the first sweep condition of `sweepSubsonic.txt`:

```
./newCase.sh □/runs/m08-a00-h0 --mesh □/meshes/h0 --solver pimple --alpha 0 \
    --Minf 0.8 --pInf 47509 --Tinf 266.17
cd □/runs/m08-a00-h0 && ./Allrun
```

3. Compare the number of cells, C_D and t_1 with the reference values of the technical report. A difference larger than the stop tolerance shows a different geometry, mesh or setup.

3.4 Prepare the computer for a study

The cost of a run is a measurement only if the computer runs at a known, constant state (Section 9.3). Before a study on a native installation:

1. Fix the CPU frequency policy: the `performance` governor or platform profile. A power-saving policy lowers the clock and changes the cost.
2. Let `perf` count events: `kernel.perf_event_paranoid` ≤ 1 for the cycles, ≤ 0 also for the energy. Make the setting permanent in `/etc/sysctl.d/`.
3. Prevent suspension during the study (power settings, or `systemd-inhibit`).
4. Give the memory and the CPU sufficient cooling. Above its temperature limit, memory refreshes more often and loses bandwidth; a CPU lowers its clock. The record of each run gives the highest temperatures (Table 9.4).
5. Run one case on each CPU socket, with nothing heavy beside it. Runs on one socket share the memory bandwidth.

3.5 Build this guide

The guide needs LuaLaTeX, BibTeX and the TeX Gyre fonts (TeX Live). The diagrams are in the repository as PDF files (`figures/pdf/`), so the guide compiles without other tools.

1. Go to `docs/guide/` and compile with `make`.
2. Only if you change a diagram: edit `figures/src/<name>.mmd` (Mermaid), and render it with `npm install` once, then `make figures`. This needs Node.js, the npm package `pele` and Google Chrome or Chromium (set `CHROME` to its executable if it is not `google-chrome`).
3. Commit the source, `figures/pdf/` and the PDF together.

Chapter 4

Geometry

4.1 Requirements of the surface

Table 4.1: Requirements of the STL file.

Item	Requirement	Reason
Format	ASCII STL	Only ASCII STL keeps the solid names, which become the patches.
Solid names	`nosecone`, `body`, `boattail`, `fins`	The dictionaries, the boundary conditions and the force integration use the patch names. `Allmesh` renames the first three to `cone`, `walls`, `tail`.
Units, position	Metres; nose tip at the origin; axis along $+x$	The refinement regions and the moment reference are written in this frame.
Edge files	`<name>_finEdges.obj`, `<name>_finRoots.obj`	Lines for the edge refinement of the fins (Section 5.2).
Topology	Closed shells, normals out	`cartesianMesh` classifies inside and outside from the surface.

A solid with another name does not stop `Allmesh`, but the patch then gets no refinement, no boundary condition and no force integration. Examine the names of a new file:

```
grep -E '^ *solid ' mesh/stl/<file>.stl
```

4.2 Design rules of the surface

`mesh/makeRocket.py` applies these rules. Each one prevents a known failure of the layer extrusion (Section 5.3).

Table 4.2: Design rules of the surface.

Rule	Principle
Round all external edges of the fins (leading edge, trailing edge, tip) and the base edge.	At a sharp edge the layers extruded from the two adjacent faces meet and tangle. A radius larger than the layer stack lets the layers turn around the edge.
A fillet at each fin root.	At a concave corner of approximately $90°$, the layers of the fin and of the body converge to one node. The fillet replaces the corner with a curve that the two layer stacks can follow.
Fins and body in one shell; the fin root cut into the body.	With separate shells the junction is not an edge of the surface, and the layers of the two shells meet at one node.
Nose and body as two closed shells on coplanar caps; the shoulder rounded.	A sharp shoulder in one continuous hull gives negative cell volumes. The rounded shoulder lets the boattail stay in the body shell, which the fillet of the trailing edge needs.
Facets around the axis: as few as give a smooth surface.	More facets give more small faces where the shells meet, and more bad cells.

The radius of an edge is set in the section parallel to the axis. Normal to a swept edge it is smaller by the cosine of the sweep angle.

4.3 Write the geometry

```
python3 mesh/makeRocket.py mesh/stl/<name>.stl [--finRoll <deg>] [--dims <file>] [
    options]
```

The script writes the STL file and its two edge files. It writes them only if the surface is closed and all normals point out. `python3 mesh/makeRocket.py -h` gives the dimensions and their defaults; `--dims` reads them from a file with one `name value` on each line.

For a half mesh, turn the fins so that the plane $z = 0$ is a plane of symmetry of the fins (`--finRoll`). The STL file always describes the whole rocket, because the half mesh is cut from the full mesh (Section 5.5).

4.4 Use a new geometry

1. Write the STL file and its edge files with `makeRocket.py`, or put them in `mesh/stl/`.
2. Examine the file against Tables 4.1 and 4.2.
3. Write the reference values and the fin data of the new rocket in `mesh/meshInfo` (Section 5.7). The refinement regions scale with `lBody` and `rBody`.
4. Examine the domain limits in `Allmesh` against Section 5.4.

5. Make a new mesh. Remove the meshes of the old geometry: `runMesh.sh` does not compare the geometry.

Chapter 5

Mesh

5.1 Octree principle

`cartesianMesh` (cfMesh) makes a hexahedral octree mesh from one maximum cell size, `maxCellSize`. Every other size is a refinement level ℓ:

$$\Delta_\ell = \texttt{maxCellSize} \,/\, 2^\ell.$$

A level acts only in its patch band or its region, and adjacent cells differ by one level at most. Two consequences follow:

- A wrong level changes only its own region.
- A change of `maxCellSize` alone scales every cell size of the mesh by the same ratio. This makes a systematic mesh series for the convergence study (Section 7.1.2).

5.2 Refinement regions

Figure 5.1 shows the nesting of the regions; each box inside another has a finer level. File: `mesh/system/meshDict`. Each mesh directory keeps its own copy.

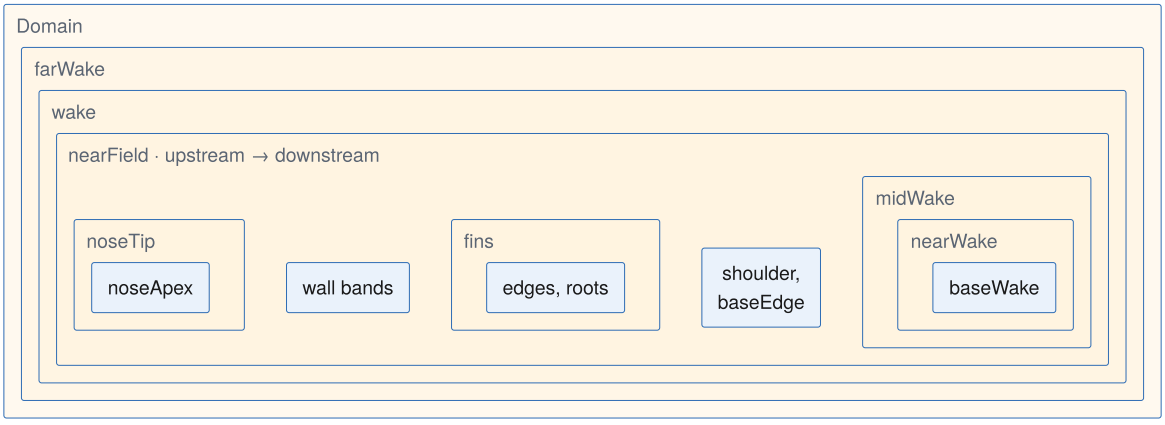


Figure 5.1: Nesting of the refinement regions, upstream to downstream.

Table 5.1: Function of each refinement and the principle that sets its level.

Refinement	What it resolves, and the principle
`localRefinement`: `cone`, `walls`, `tail`	A band of thickness `refinementThickness` along each wall. It contains the boundary layer and, on the nose, the region between the wall and the bow shock. The surface cell size also sets the layer thickness (Section 5.3).
`localRefinement`: `fins`	The fin chord. A finer level than the body, because the fin is thin and its edges carry the strongest gradients.
`edgeMeshRefinement`: `finEdges`	The leading edge (stagnation point, attached or detached shock) and the trailing edge (expansion).
`edgeMeshRefinement`: `finRoots`	The fin-body junction, where the boundary layers of two walls meet.
`noseTip`, `noseApex`	The curvature of the nose tip: the cell must be smaller than the local radius of curvature, and the layers thinner than it.
`shoulder`, `baseEdge`	The expansion over the boattail and the separation at the base edge, where the base shear layer starts. They set the base pressure.
`baseWake`, `nearWake`, `midWake`	The base recirculation and the shear layer, one level coarser in each region downstream, as the wake spreads and its gradients decrease.
`nearField`, `wake`, `farWake`	Transition from the body to `maxCellSize`, so that the level changes gradually.

The regions are written as fractions of the body: each coordinate along x is multiplied by `lBody`, and each coordinate across by `rBody`, both from `meshInfo`. The regions thus follow a rocket of another size.

The wake regions are cones on the body axis. At an angle of attack α the wake moves by $\tan\alpha$ for each unit length downstream. Make sure that the wake stays inside the wake regions over the length that sets the base pressure, for the largest angle of the study.

CAUTION

`foamDictionary -set` evaluates the `#eval` entries of `meshDict` and writes numbers. `Allmesh` writes `lBody` and `rBody` with `sed` first, for this reason. Do not set `lBody` with `foamDictionary`.

5.3 Boundary layers

cfMesh extrudes the layers from the surface mesh. The thickness of the first cell t_1 thus comes from the surface cell size h, the number of layers n (`nLayers`) and the growth ratio r (`thicknessRatio`):

$$t_1 \propto h\,\frac{r-1}{r^n-1}.$$

`cartesianMesh` makes the layer thinner than this rule by a constant factor, so the rule gives equal t_1 on meshes with equal right-hand side, but not the value of t_1.

Principle. In a mesh study, keep t_1 equal on all meshes. If t_1 changes, y^+ changes, and the study mixes the change of the wall treatment with the discretization error. As h decreases from mesh to mesh, decrease n or adjust r so that the rule gives the same value. `meshes.txt` gives n and r for each mesh. Measure t_1 on each new mesh:

```
./post/firstLayer.py <mesh-dir>
```

It gives y, the distance from the centre of the first cell to the wall (as `yPlus` uses it), and $t_1 = 2y$, for each wall patch, weighted by the face area.

Estimate of y^+. The mesh does not set y^+. Estimate it before a study from the flat-plate friction at the aft body, where y^+ is lowest:

$$Re_x = \frac{U_\infty x}{\nu_\infty}, \qquad C_f \approx 0.0576\, Re_x^{-1/5}, \qquad u_\tau = U_\infty \sqrt{C_f/2}, \qquad y^+ = \frac{y\, u_\tau}{\nu}.$$

y^+ grows with the speed, so one mesh gives the first cell in the viscous sublayer at low Mach numbers and in the buffer or log layer at high Mach numbers. The wall function of ν_t must then be valid for all y^+ (Section 8.6.1). Measure y^+ on each run (Section 9.1).

Layers only on wall patches. cfMesh splits the layer only on the patches with an entry in `patchBoundaryLayers`. The global `nLayers` is for the domain faces. Keep an entry for each wall patch.

Fins. The fins have fewer layers than the body (`patchBoundaryLayers/fins`). A thin fin cannot hold a thick layer stack: the layers of the two faces meet. `allowDiscontinuity` lets the number of layers change at the fin-body junction.

5.4 Domain

`Allmesh` sets the domain with `surfaceGenerateBoundingBox`, from distances to the bounding box of the rocket. The upstream and downstream distances are fixed in `Allmesh`; the lateral walls are at a fixed distance from the axis for any rocket, so the box is symmetric about $z = 0$.

Boundary	Principle
Upstream	In subsonic flow the disturbance of the body decays with the distance; the inlet must be far enough that the free-stream condition is correct there. In supersonic flow the inlet must be upstream of the bow shock.
Lateral	In subsonic flow, small blockage. In supersonic flow, the Mach cone from the nose, at the angle $\mu = \arcsin(1/M)$, exits through the lateral boundary, so that boundary needs a non-reflecting condition (Section 8.8).
Downstream	The outlet must be far behind the base recirculation, so that no flow enters through it.

CAUTION

`Allmesh` calculates the lateral distances with `awk` and sets `LC_ALL=C`. In a locale with a decimal comma, `awk` writes numbers that OpenFOAM reads incorrectly (Section 13.3).

5.5 Half mesh

If the fins have a plane of symmetry that contains the free-stream velocity, half the domain is sufficient. By default `Allmesh` turns the fins so that $z = 0$ is such a plane, meshes the whole rocket, and keeps the half $z \geq 0$:

1. The domain box is symmetric about $z = 0$, so the octree has faces on that plane and the cells do not cross it.
2. `topoSet` and `subsetMesh` keep the cells with the centre at $z \geq 0$.
3. `snapPlane.py` moves the points of the cut onto $z = 0$. The smoothing of `cartesianMesh` leaves them slightly off the plane.
4. `createPatch` makes the cut the patch `symm`, of type `symmetry`.
5. `meshInfo` gets `sector half` and half the reference area, so the coefficients are those of the full rocket. The flow can then have an angle of attack in the x–y plane, but no sideslip.

CAUTION

Do not mesh the half domain directly, with a domain face on $z = 0$. `cartesianMesh` treats that face as a domain boundary that touches the wall: it extrudes wall layers into it, and the layers fan out at the corner.

5.6 Patches

`createPatch` merges the six faces of the box into `inlet`, `outlet` and `box`, of type `patch`. The `.fms` surface gives no type to its patches, so `cartesianMesh` makes all of them `wall`; after `createPatch`, only the rocket patches stay `wall`. The wall functions and the `yPlus` function object need this type. `Allmesh` stops if a patch in `wallPatches` is not `wall`.

Feature edges. `surfaceFeatureEdges` marks an edge as a feature edge when the angle between the two adjacent triangles exceeds a limit. `cartesianMesh` moves vertices onto these edges. Set the limit above the facet angle of the rounded edges, so that only the real edges of the box and the nose are features. A low limit marks false edges where the facets of a round are small, and the cells bend along them.

5.7 meshInfo

File: `mesh/meshInfo`. `Allmesh` copies it into the mesh directory and adds the record of the mesh; `newCase.sh` copies it into the case.

Table 5.2: Entries of `meshInfo`.

Entry	Written by	Function
`sector`, `symmetryFraction`	user, `Allmesh`	Part of the rocket in the mesh. `Allrun` checks the flow angles against it.
`Aref`, `lRef`	user, `Allmesh`	Reference area (scaled with the sector) and length of the coefficients.

Entry	Written by	Function
`rBody`, `lBody`	user	Scale the refinement regions.
`fins`, `finRootR`, `finTipR`, `finX0`, `finX1`	user	Fin data for `Allrefine`.
`wallPatches`	user	Patches in the force integration.
`nCells`, `maxCellSize`, `finLevel`, `nLayers`, `thicknessRatio`, `meshDir`	`Allmesh`	Record of the mesh.

5.8 Make a mesh

```
mesh/Allmesh <mesh-dir> [stl-file] [--maxCellSize <m>] [--nLayers <n>]
             [--thicknessRatio <r>] [--full]
```

The options write into the copy of `meshDict`; the options that you do not give keep the values of `mesh/system/meshDict`. Without `--full` the mesh is a half mesh. `Allmesh` does not write into a directory that exists.

`Allmesh` reads the log of each application and stops on `FOAM FATAL`, because `runApplication` returns 0 also when an application fails.

5.9 Examine a mesh

1. Read the summary of `Allmesh`. A mesh with negative cell volumes is not usable: change the geometry or `meshDict`, and make a new mesh.
2. Examine the patch types in `constant/polyMesh/boundary`.
3. Measure t_1 with `post/firstLayer.py`, and compare it with the other meshes of the study.
4. Look at the cells on the fin edges, the base edge and the boattail shoulder in ParaView.

NOTE

`checkMesh -allGeometry` also reports small determinants in layer cells, face tets and concave cells. These checks are of a different category from negative volumes and do not stop the use of the mesh. Record them with the mesh.

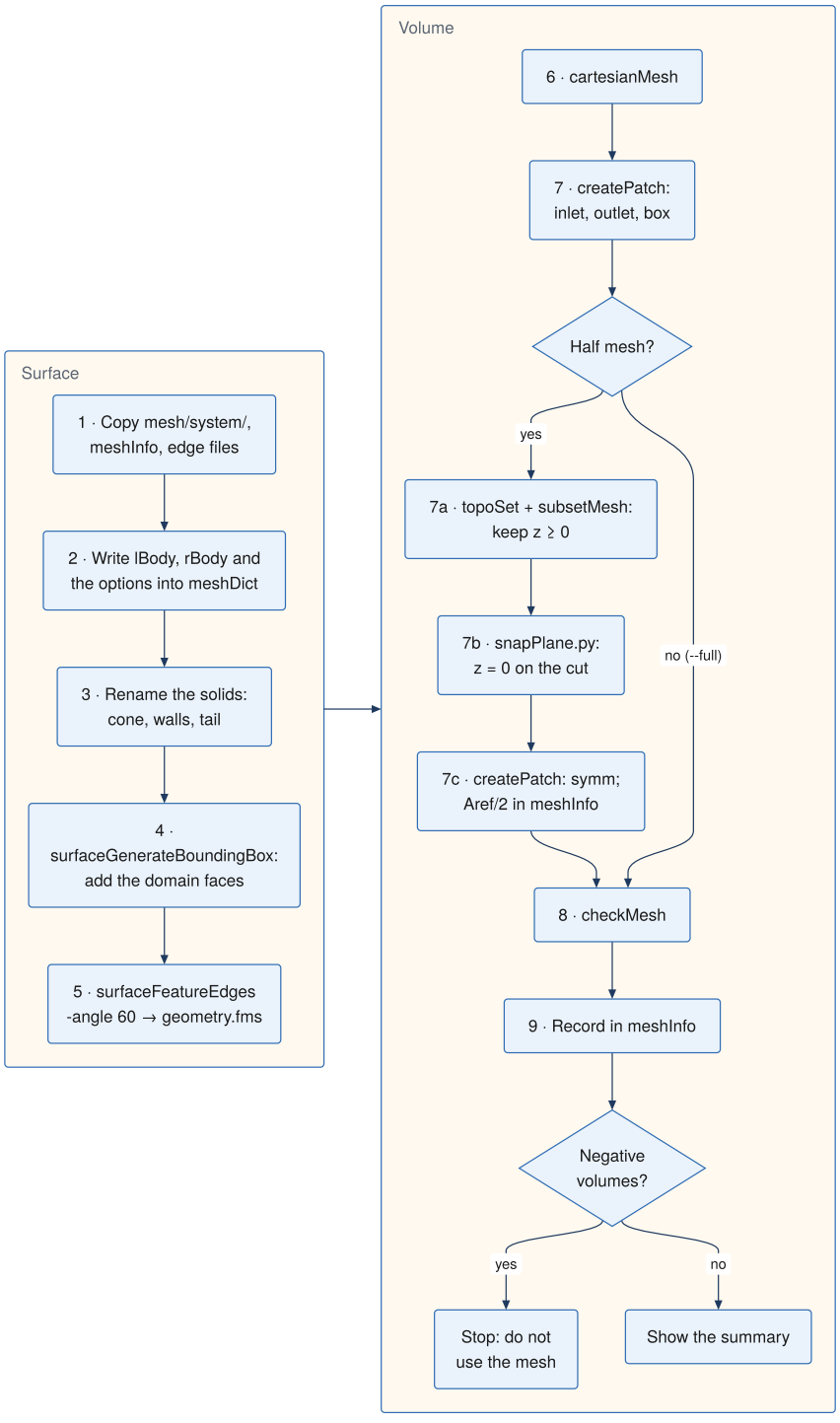


Figure 5.2: Steps of `Allmesh`: surface operations, then volume operations.

Chapter 6

Cases

A case is one run: one template, one mesh and one flight condition. Make it with `newCase.sh` and run it with `Allrun`. `runCase.sh` does the two steps for each line of a sweep file (Chapter 7).

6.1 Make a case

6.1.1 Command

```
./newCase.sh <run-dir> --mesh <mesh-dir> [options]
```

Table 6.1: Options of `newCase.sh`.

Option	Function
`--mesh <mesh-dir>`	Mandatory. The path of a mesh that `Allmesh` made.
`--Minf <M>`	Free-stream Mach number.
`--pInf <Pa>`, `--Tinf <K>`	Free-stream static pressure and temperature.
`--alpha <deg>`, `--beta <deg>`	Angle of attack and angle of sideslip.
`--Ti <->`, `--nuRatio <->`	Free-stream turbulence intensity and ratio ν_t/ν.
`--np <N>`	Number of subdomains in `system/decomposeParDict`.
`--socket <S>`	Binds the run to socket `S`, one process per physical core (Section 6.2.3). `none` disables the binding.
`--refine tip`, `--refine fins`	Runs `Allrefine` on the new case (Section 6.4).
`--init <run-dir>`	Starts from the latest time of another run (Section 6.3).
`--solver central`, `--solver pimple`	Template `case-central/` (default) or `case-pimple/`.

The flow options change entries in `system/flowConditions`. The options that you do not give keep the values of the template (Section 8.3).

NOTE

For `newCase.sh`, `--mesh` and `--init` are paths. Only `runCase.sh` accepts a name without a path.

6.1.2 Steps that newCase.sh does

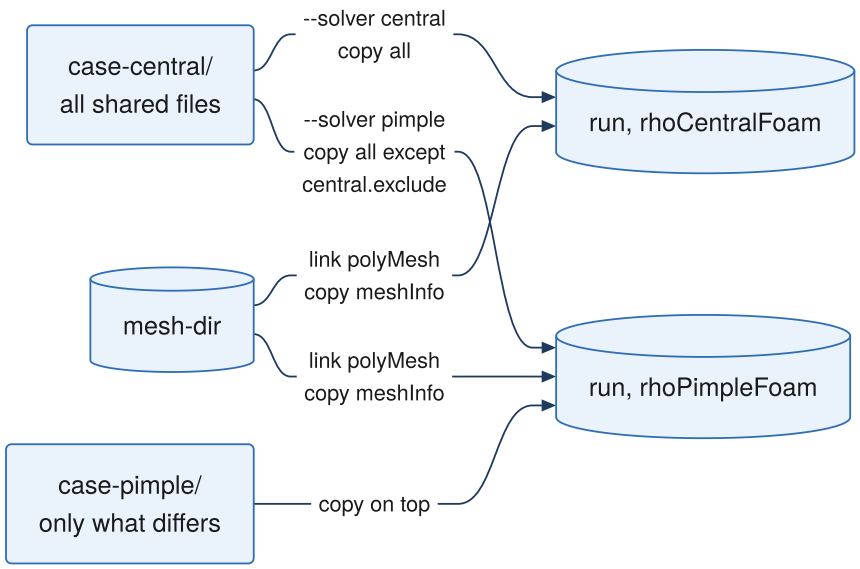


Figure 6.1: How `newCase.sh` assembles a case from the templates and a mesh.

1. It stops if `--mesh` is missing, if the mesh has no `constant/polyMesh`, or if `<run-dir>` exists.
2. It copies `case-central/` into `<run-dir>`, without the mesh, the logs, `0/`, `processor*`, `postProcessing/` and the time directories. With `--solver pimple`, it leaves out the files in `case-pimple/central.exclude` and then copies `case-pimple/` on top (Figure 6.1).
3. It links `constant/polyMesh` to the mesh. With `--refine` it copies it.
4. It copies `<mesh-dir>/meshInfo` to `constant/meshInfo`.
5. It copies `common/Allrun`, `common/Allrefine` and `common/machineLog.py` into the case.
6. It writes `run.info` with the case name, the date, the commit of the repository, the mesh, the number of cells and the options.
7. It renders each `*.j2` file with the values in `config.json` (Section 8.5.3), and removes the `*.j2` file.
8. It writes each flow option into `system/flowConditions`. It stops if the entry does not exist.
9. It writes `--np` into `decomposeParDict` and `--socket` into the file `socket`. With `--init`, it writes the path of the other run into `initFrom` and into `run.info`.
10. With `--refine`, it runs `Allrefine`.
11. It adds to `run.info` the fingerprint of the mesh (`meshHash`) and the physical definition of the case (block `flow`, Section 10.1).

6.2 Run a case

6.2.1 Command

```
cd <run-dir>
./Allrun        # parallel, numberOfSubdomains from system/decomposeParDict
./Allrun 8      # parallel on 8 processes; writes 8 into decomposeParDict
./Allrun 1      # serial
```

6.2.2 Steps that Allrun does

Figure 6.2 shows the steps.

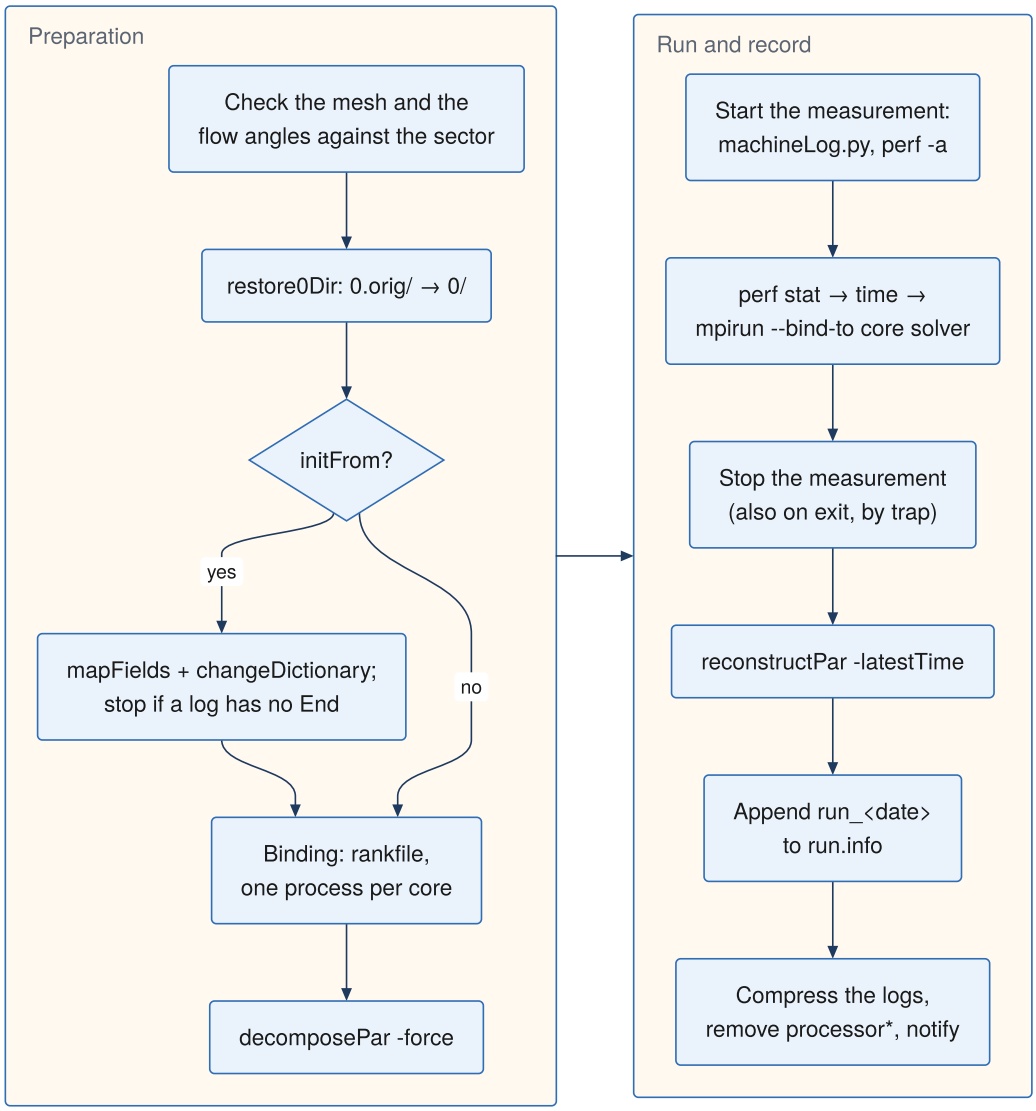


Figure 6.2: Steps of `Allrun`.

1. It stops if `constant/polyMesh` does not exist.
2. It compares `alpha` and `beta` in `flowConditions` with `sector` in `meshInfo` (Table 6.2). It stops if the mesh cannot show the flow angle.
3. It copies `0.orig/` to `0/`. If `initFrom` exists, it maps the other run onto `0/` (Section 6.3).

4. It sets the binding of the processes (Section 6.2.3).
5. It runs `decomposePar -force`.
6. It starts the measurement (Section 9.3) and runs the solver with `mpirun`. It stops the measurement when the solver ends. A trap also stops it if `Allrun` stops early.
7. It runs `reconstructPar -latestTime`, and removes `processor*` if that step ends correctly.
8. It adds a record `run_<date>` to `run.info` (Section 9.2), compresses the logs with `xz` and sends the notification (Section 9.7).

Table 6.2: Flow angles that each mesh sector permits.

`sector`	Permitted angles
`quarter`	$\alpha = 0$ and $\beta = 0$
`half`	$\beta = 0$ (the default mesh)
`full`	All angles

6.2.3 Process binding

`Allrun` binds each MPI process to one physical core when:

- the case has a file `socket` with a socket number, or
- the computer has one socket and the number of processes is not more than the number of physical cores.

It writes a `rankfile` with process i on core i of the socket, and runs `mpirun --bind-to core`. It stops if the processes are more than the cores of the socket. To run without binding, for example a short test beside a sweep, write `none` in the file `socket`.

CAUTION

`Allrun` does not use the OpenFOAM function `runParallel`, because it always adds `--oversubscribe`. With that option, Open MPI 5 ignores the `rankfile`, and each process can move to all CPUs. `run.info` then shows a binding that did not occur. To check the binding of a run, examine each process: `taskset -cp <pid>` must give the two threads of one core.

6.2.4 End of a run

The run stops at `endTime`, or when the stop criterion finds that the mean of C_D is constant (Section 8.5.4). The record in `run.info` gives `status ok` if the solver log ends with `End`.

6.2.5 Continue a run

`controlDict` has `startFrom latestTime`. Run `./Allrun` again in the same directory to continue from the last time directory. The continuation adds a new record to `run.info` and writes the function objects into a new directory with its start time.

CAUTION

`Allrun` runs `decomposePar -force`, which removes the `processor*` directories. If the solver stopped before `reconstructPar`, run `reconstructPar -latestTime` before you run `./Allrun` again. If you do not, you lose the data after the last reconstructed time.

On the first continuation, read the start time in the solver log. Make sure that it is the last time and not 0.

6.2.6 Start a run again from zero

1. Remove the time directories and the processor directories: `rm -rf [1-9]* processor*`.
2. Run `./Allrun`.

`./Allclean` also removes `constant/polyMesh`. After `./Allclean`, make a new case with `newCase.sh`.

6.3 Start from another run

`newCase.sh --init <run-dir>` starts the case from the solution of another run instead of the uniform free stream. The sweep uses it from a coarse mesh to the next finer mesh at the same flight condition, as the OpenFOAM ToolChain does [8, §5.4]. The fine mesh starts with the shock, the wake and the boundary layer in place. The two runs can use different solvers. After `restore0Dir`, `Allrun` does these steps:

1. It reads the value of each `fixedValue` patch in `0/`. These are the values of this run: in a supersonic case, `p` on `inlet`.
2. It runs `mapFields <run-dir> -consistent -sourceTime latestTime`. This maps the internal field and the patch values. The other run can have another mesh of the same domain and patches.
3. It runs `changeDictionary` to write back the values of step 1.
4. It stops if the log of `mapFields` or of `changeDictionary` does not end with `End`.
5. It deletes `0/rho`. The solver calculates `rho` again from `p` and `T`.

CAUTION

A cell of this mesh that `mapFields` does not find in the other mesh keeps the value of `0.orig`; such cells occur where the two meshes differ near the wall, for example at the fin tips. `0.orig` has no `rho`, so these cells get a density equal to the numerical value of the pressure. `rhoPimpleFoam` reads `0/rho` if it is there, and the run fails in the first steps. For this reason `Allrun` deletes `0/rho` after the mapping. Apply the same rule to any field that the solver calculates from others.

The free-stream entries of the patches (`freestreamValue`, `inletValue`, `fieldInf`) are not mapped. They keep the values of this run. The other run must have a time directory with the solution.

CAUTION

`mapFields` can stop with a segmentation fault when the source and the target are the same mesh, which is the case of the supersonic start. `Allrun` then stops the run (Section 13.1).

6.4 Refine the mesh near the fins (optional)

```
./Allrefine          # same as ./Allrefine tip
./Allrefine tip      # cell set finTip: a band around the fin tip
./Allrefine fins     # cell set finBox: the full fin, from root to tip
```

Do this procedure in the case directory, before `./Allrun`, or give `--refine` to `newCase.sh`. `Allrefine` does these steps:

1. It stops if `0/U` exists. `refineMesh` maps the fields, so a refinement after a run changes the results.
2. It stops if `meshInfo` does not have `fins true` and `finTipR`.
3. It replaces the link to the mesh with a copy, so that it does not change the shared mesh.
4. It runs `topoSet`. `system/topoSetDict` makes the cell sets from the fin geometry in `meshInfo`.
5. It runs `refineMesh`. Each cell in the set becomes 8 cells.
6. It runs `checkMesh` and `renumberMesh`, and writes the new number of cells into `meshInfo`.

Set	Region	Effect on y^+
`finTip`	Annulus from `finTipR` − `tipBand` to `finTipR` + `margin`	None. The region does not touch the body.
`finBox`	Cylinder of radius `finTipR` + `margin`	The first body cell below the fins is half as thick, so y^+ there is approximately half.

Chapter 7

Studies and sweeps

A study is a set of runs: flight conditions × angles of attack × meshes. `runMesh.sh` makes the meshes; `runCase.sh` makes and runs the cases on the meshes that exist.

7.1 Design of a study

Dimension	Principle
Mach numbers	Points of the flight where the drag must be known, with more points where C_D changes fast (the transonic rise).
Flight conditions	p_∞ and T_∞ of the real atmosphere at the height where the vehicle passed each Mach number, so that the Reynolds number is that of the flight (Section 7.4).
Angles of attack	From zero to the largest angle that the flight reached, with margin. Measure the flight angle when possible (Section 11.3).
Meshes	At least three meshes of one systematic series for the error estimate, plus a coarser mesh to start the runs (Section 7.1.2).

7.1.1 Chains of runs

Each flight condition and angle is a chain: each run starts from the solution on the coarser mesh (`--init`, Section 6.3). The fine mesh then starts with the shocks, the wake and the boundary layer in place, and skips the start transient, which is the most expensive and least stable part of a run [8, §5.4].

A supersonic chain starts with a `rhoPimpleFoam` run on the coarsest mesh, because `rhoCentralFoam` from a uniform free stream fails at the base (Section 8.12).

7.1.2 Mesh series for the convergence study

1. Change only `maxCellSize`, with a constant ratio, and keep all levels. All cell sizes then change by the same ratio.

2. Use a ratio that gives a refinement ratio of more than 1.3 in the representative cell size, as the method of Celik et al. [9] needs. An octree mesh does not refine uniformly: calculate the real ratio of each pair from the number of cells, $r = (N_{\text{fine}}/N_{\text{coarse}})^{1/3}$, and use it in Section 12.4.

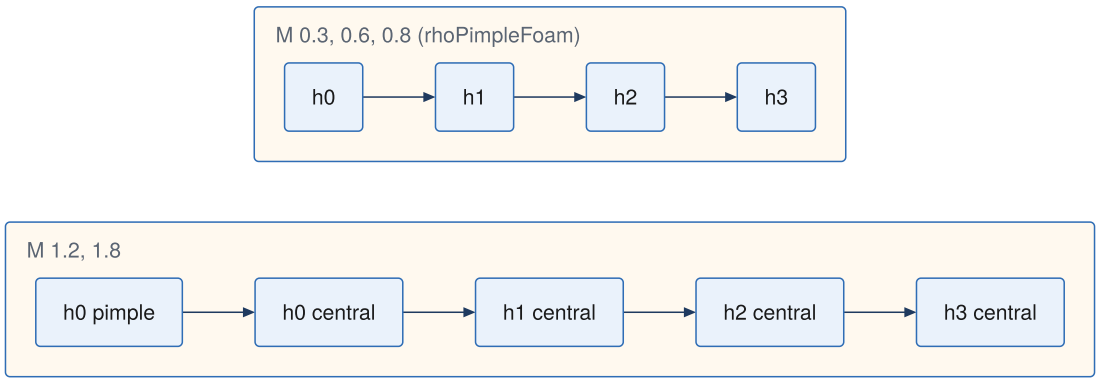


Figure 7.1: Chains of runs. Left to right: from coarse to fine. Each arrow is an `--init`.

3. Keep t_1 equal on all meshes (Section 5.3).
4. Run the same flight conditions on all meshes.

7.2 Make the meshes

`meshes.txt` has one mesh on each line: a name, then the options of `Allmesh`. Lines that start with # have no effect.

```
./runMesh.sh [meshes-file] [meshes-dir]      # default: meshes.txt, /meshes
```

For each line: if the mesh exists with the same options, `runMesh.sh` keeps it; if the options differ, it stops; if not, it runs `Allmesh`. It does not compare the geometry: after a change of the STL file, remove the old meshes. To make one mesh only, give a file with that line.

7.3 Run the cases

A sweep file has one case on each line: a name, then the options of `newCase.sh` (Table 6.1). A mesh or an `--init` without `/` is a name in `<meshes-dir>` or `<runs-dir>`. The run of an `--init` must come earlier in the file.

```
m08-a00-h1  --mesh h1 --solver pimple --alpha 0 --Minf 0.8 --pInf <Pa> --Tinf <K> --
    init m08-a00-h0
```

```
./runCase.sh [--np <N>] [--socket <S>] <sweep-file> [runs-dir] [meshes-dir]
```

The defaults are the physical cores of one socket, `~/runs` and `~/meshes`. The sweep files do not contain the number of processes, so the same file works on any computer. An option on a line of the file overrides the option of `runCase.sh`.

For each line, `runCase.sh` skips the case if its directory exists, stops if the mesh does not exist, runs `newCase.sh`, and runs `Allrun`. A failed run does not stop the sweep. Thus, after a stop, run `runCase.sh` again: it continues from the first case that does not exist.

Processes and sockets. One process for each physical core of one socket, bound to its core (Section 6.2.3), is the default. On a computer with two sockets, run `sweepSubsonic.txt` with

`--socket 0` and `sweepSupersonic.txt` with `--socket 1` at the same time. Each chain must stay in one file, because the files run in parallel.

7.3.1 Start a sweep

1. Do the steps of Section 3.4.
2. Start the sweep so that it continues when the terminal closes:

```
nohup ./runCase.sh <sweep-file> > □/log.<sweep> 2>&1 &
```

3. On the first case, examine the binding (Section 6.2.3) and the measurement files (Section 9.3).

7.3.2 Stop a sweep

1. Stop `runCase.sh` by its process number (`kill <pid>`). No new case starts; the current solver continues.
2. To stop the solver also, stop its `mpirun` (or `prterun`) by its process number. `Allrun` then reconstructs the last time, writes the record and stops the measurement.

CAUTION

`runCase.sh` reads the sweep file while it runs. Do not edit the file in place until the sweep stops.

7.4 Flight conditions

The conditions of each Mach number come from the flight telemetry (Chapter 11): the ambient pressure and temperature at the height where the vehicle passed that Mach number in the coast phase, with the atmosphere of the day of the flight. The standard atmosphere can differ by tens of kelvin from the real one, which changes the Mach number at a given speed, the density and the Reynolds number. `cdFlight.py` prints the conditions; write them into the sweep files.

7.5 Rules

- Do not put meshes or runs in the repository. Put the STL file, `meshDict`, `meshes.txt` and the sweep files in it.
- When a change to `meshDict` changes the mesh, commit it with the `checkMesh` summary of the new mesh.

Chapter 8

Solvers and models

This chapter gives the principle behind each choice of the templates and the file where each value is. Read the values in the files.

8.1 Two solvers

	`case-pimple/`: `rhoPimpleFoam`	`case-central/`: `rhoCentralFoam`
Formulation	Pressure-based, `transonic` pressure equation	Density-based, explicit, central-upwind flux
Use	$M < 1$; start of each $M > 1$ chain	$M > 1$

The numerical dissipation of a central-upwind flux scales with the speed of sound, not with the flow speed. At low Mach numbers it is thus large compared with the convective flux, and the drag error grows. A pressure-based solver does not have this problem, but it loses accuracy where strong shocks occur. The OpenFOAM ToolChain report [8, §6] measured both effects on rockets. The workflow therefore changes solver at $M = 1$.

CAUTION

The change of discretization at $M = 1$ can cause a step in $C_D(M)$ that is not physical. To measure it, run one Mach number with both solvers on the same mesh.

The models, the gas, the boundary conditions, the function objects and the flight condition are the same files in the two templates: `case-pimple/` holds only the files that differ (Section 8.11).

8.2 Files of the templates

Table 8.1: Files of the templates and their function.

File	Function
`system/flowConditions`	The flight condition: the only input of a case (Section 8.3).
`system/flowDerived`	Values derived from `flowConditions` with `#eval`.

File	Function
`config.json`	Courant ramp and stop criterion; `newCase.sh` renders the `*.j2` files with it.
`0.orig/`	Boundary and initial conditions (Section 8.8).
`constant/thermophysicalProperties`	Gas model (Section 8.7).
`constant/turbulenceProperties`	Turbulence model (Section 8.6).
`system/controlDictBase.j2`, `FOs/FOcommon`	Time control and function objects (Section 8.9).
`system/fileUpdater.j2`, `fileUpdaterControlDict/`	The Courant ramp of the central template (Section 8.5.2).
`system/FOs/FOrunTimeControl.j2`	Stop criterion (Section 8.5.4).
`system/fvSchemes*`, `system/fvSolution`	Schemes and linear solvers (Section 8.10).

CAUTION

Use `foamDictionary -set` only on `flowConditions`. On a file with `#include` or `#eval`, it writes the expanded values, and the file stops following `flowConditions`.

8.3 Flight condition

`flowConditions` holds M_∞, p_∞, T_∞, α, β, the turbulence intensity T_i and the ratio ν_t/ν of the free stream. `flowDerived` calculates from them:

$$a_\infty = \sqrt{\gamma R T_\infty}, \quad U_\infty = M_\infty a_\infty, \quad \rho_\infty = \frac{p_\infty}{R T_\infty}, \quad \mu_\infty = \frac{A_s \sqrt{T_\infty}}{1 + T_s/T_\infty},$$

$$k_\infty = \tfrac{3}{2}(U_\infty T_i)^2, \quad \nu_{t\infty} = \frac{\nu_t}{\nu}\,\nu_\infty, \quad \omega_\infty = \frac{k_\infty}{\nu_{t\infty}},$$

and the velocity, drag and lift directions from α and β (wind axes). The free-stream turbulence is small and the eddy-viscosity ratio of order one, so that the free stream does not change the boundary layer.

8.4 Numerical method of rhoCentralFoam

`rhoCentralFoam` advances the conserved variables ρ, $\rho\mathbf{U}$ and ρE explicitly, then corrects the diffusive terms implicitly. It solves no pressure equation.

Flux. The central-upwind scheme of Kurganov [10, 11] weights the face flux with the local one-sided wave speeds. It captures shocks without a Riemann solver and without artificial viscosity. Greenshields et al. [12] give its implementation in OpenFOAM.

Reconstruction. The primitive variables go to the faces with the van Leer TVD limiter [13]: second order in smooth regions, first order at a discontinuity, so no new extrema occur at shocks.

8.5 Local time step, Courant ramp and stop criterion

8.5.1 Local time step

The templates solve a steady problem with a pseudo-transient march (`ddt localEuler`). Each cell advances with the largest time step that its own Courant number permits. The intermediate states are not physical; the converged state is. The cost then depends on the number of iterations, not on the smallest cell of the mesh, which with a global step sets the step of the whole domain.

`deltaT` is 1, so the time counts iterations: `endTime`, the write interval, the ramp and the stop criterion are in iterations. The central template reads the local-step controls in `controlDict`, the pimple template in the `PIMPLE` dictionary of `fvSolution`.

CAUTION

With `localEuler`, the Courant number that `rhoCentralFoam` writes uses the global `deltaT`, not the local step: it is meaningless. Read the line `Flow time scale min/max`: its minimum is the step of the most restrictive cell, and a sudden decrease shows a cell that diverges.

8.5.2 Courant ramp (central template)

A uniform free stream on a no-slip wall is a strong impulsive start. The explicit central solver has no damping of the local step, so a large Courant number at the start makes the solution leave the physical range. The ramp [8, §5.4.1] starts with a small Courant number and first-order reconstruction, increases the Courant number step by step, then changes to second order and ramps again (Figure 8.1). The function objects `timeActivatedFileUpdate` copy a new `controlDict` or `fvSchemes` at each switch; `runTimeModifiable` makes the solver read it.

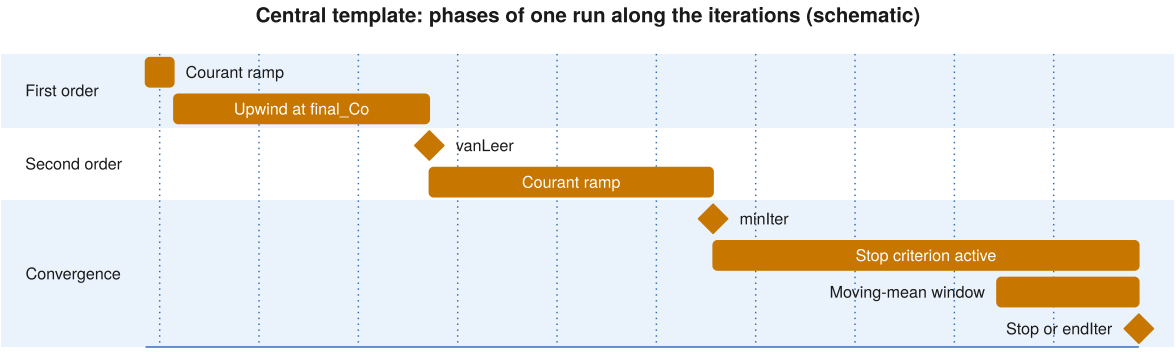


Figure 8.1: Phases of a run of the central template along the iterations. The diamonds are the keys of `config.json` that place each switch.

The final Courant number never comes before the end of the ramp: a jump of the Courant number makes the explicit solver fail some iterations later. The pimple template has no ramp: its implicit pressure-velocity coupling damps the start.

8.5.3 config.json

Key	Function
`maxCo` (pimple), `final_Co` (central)	Courant number of the converged phase.
`vanLeer` (central)	Iteration of the change from first to second order.
`minIter`	First iteration of the stop criterion. In the central template, also the change to the final Courant number.
`endIter`	Last iteration if the criterion does not stop the run.
`coeffs_fields`, `coeffs_variation`, `coeffs_window`	Coefficients, absolute tolerances and window of the stop criterion.

The schedule comes from the ToolChain, which used another rocket. Adjust it from the first runs: the iteration at which C_D becomes flat in first order, and the iterations that the second order needs.

8.5.4 Stop criterion

For each coefficient C in `coeffs_fields`, `runTimeControl` calculates the moving mean $\overline{C}$ over `coeffs_window` iterations, from `minIter`. The run stops when the window is complete and

$$|\overline{C} - C| < \texttt{coeffs_variation}$$

for all coefficients. Set the tolerance as a fraction of the expected coefficient: an absolute tolerance is a smaller fraction of a large coefficient. The tolerance is absolute, so it also works for a coefficient near zero, such as C_L at $\alpha = 0$. At $\alpha \neq 0$, add C_L and C_m to the criterion, or examine that they are flat.

To change the tolerance of a running case, edit `tolerance` in its `system/FOs/FOrunTimeControl` and `touch system/controlDict`. The mean then starts again from zero.

8.6 Turbulence

RANS with the k–ω SST model [14], in the version of Menter et al. [15] that OpenFOAM uses. The Boussinesq hypothesis gives the Reynolds stress from an eddy viscosity,

$$-\overline{u_i' u_j'} = \nu_t \left(\frac{\partial U_i}{\partial x_j} + \frac{\partial U_j}{\partial x_i} \right) - \tfrac{2}{3} k\, \delta_{ij}, \qquad \nu_t = \frac{a_1 k}{\max(a_1 \omega,\ SF_2)}.$$

The shear-stress limiter in the denominator limits ν_t in adverse pressure gradients, which occur on the boattail and at the fin-body junction and set the base flow. When you compare with the literature, find which SST version the source uses.

8.6.1 Wall treatment

Field	Condition	Principle
`nut`	`nutUSpaldingWallFunction`	Spalding's law [16] is one formula from the viscous sublayer through the buffer layer to the log layer, so it is valid for any y^+ of the first cell.
`k`	`kLowReWallFunction`	Also continuous in y^+.
`omega`	`omegaWallFunction`	Blends the sublayer and log-layer values.
`alphat`	`compressible::` `alphatWallFunction`	Turbulent heat diffusivity from ν_t and Pr_t.

A y^+-insensitive wall treatment is necessary, because one mesh gives different y^+ at different Mach numbers (Section 5.3). The conditions need wall patches of type `wall`.

8.7 Gas

Air as a perfect gas ($p = \rho RT$) with constant c_p (`hConst`), energy as sensible internal energy, and Sutherland viscosity, $\mu = A_s\sqrt{T}/(1 + T_s/T)$ [17, eq. 1-36]. `rhoCentralFoam` needs `hePsiThermo` and the internal energy.

Prandtl number. With Sutherland transport, OpenFOAM calculates the conductivity with the modified Eucken correction, $\kappa = \mu\, c_v\, (1.32 + 1.77\, R/c_v)$. The laminar Prandtl number is therefore an output, not an input:

$$Pr = \frac{c_p}{c_v\,(1.32 + 1.77\,R/c_v)}.$$

For air it is lower than the usual 0.72. Another solver must use the same value (Chapter 10). The turbulent Prandtl number Pr_t is an input [18].

Validity of constant c_p. The stagnation temperature is $T_0 = T\,(1 + \frac{\gamma-1}{2}M^2)$. When T_0 is high enough that c_p of air changes by more than the accepted error, replace `hConst` with `janaf`. The `rhoCentralFoam` tutorial `biconic25-55Run35` has the air coefficients.

Gas constant. `flowDerived` and the thermophysical model must use the same R, γ and Sutherland constants. Compare them when you change one.

8.8 Boundary conditions

The number of characteristics that enter the domain through a boundary sets the number of values that the boundary may impose.

Field	`inlet`, `outlet`, `box`	Principle
`U`	`freestreamVelocity`	Fixed where the flow enters, extrapolated where it leaves.
`T`, `k`, `omega`	`inletOutlet`	The same.
`p`, $M < 1$	`freestreamPressure`	One acoustic wave enters through the outlet, so p is fixed where the flow leaves and extrapolated where it enters.
`p`, $M > 1$	`fixedValue` inlet, `zeroGradient` outlet, `waveTransmissive` box	All characteristics enter at a supersonic inlet and leave at a supersonic outlet. On the lateral boundary the Mach cone leaves the domain: a non-reflecting condition [19] prevents its reflection.

`0.orig/p` selects the branch with `#if` on `Minf`, because `freestreamPressure` has no supersonic mode in v2412.

Walls. No slip and adiabatic (`zeroGradient` on T). During a short motor burn, the airframe does not get near the recovery temperature, so zero heat flux is the correct limit.

8.9 Function objects

`system/FOs/FOcommon` is common to the two templates. Chapter 9 lists the outputs. The force coefficients use the density field (the pressure is in Pa), the free-stream density and speed for the dynamic pressure, `Aref` and `lRef` from `meshInfo`, the nose tip as the moment reference, and the wind axes.

8.10 Discretization schemes and linear solvers

Choice	Principle
TVD limiters on the convection of the flow variables	Second order without new extrema at shocks.
First-order upwind for k and ω	Keeps them positive.
Limited non-orthogonal correction	The layer cells are non-orthogonal; the limit keeps the correction bounded.
`meshWave` wall distance	Needed by the SST blending functions.
`diagonal` for ρ, $\rho\mathbf{U}$, ρE (central)	The conserved variables are explicit.
Smooth solvers for $\mathbf{U}$, e, k, ω; GAMG for p (pimple)	An elliptic pressure equation needs a multigrid solver.

In the central template, `fvSchemes` and `fvSchemesUpwind` include `fvSchemesVanLeer` and set the reconstruction to upwind; the ramp copies them. Do not edit `system/fvSchemes`.

8.11 rhoPimpleFoam template

`case-pimple/` holds `config.json`, `controlDict.j2`, `fvSchemes`, `fvSolution.j2`, `fvOptions` and `central.exclude`. The schemes come from the transonic template of the ToolChain.

Choice	Principle
`transonic yes`	A density-weighted pressure equation for the compressibility of high subsonic flow.
One outer corrector	A pseudo-steady march needs one pressure-velocity pass for each iteration.
Bounded gradients and TVD convection	Second order away from the local shocks of transonic flow.
`pMinFactor`, `pMaxFactor`	Limits of p relative to the boundary values; the upper limit must contain the stagnation pressure.
`limitTemperature` in `fvOptions`	Protects the start transient. Count the limited cells in the log: after the start there must be none.

8.12 Start of the supersonic cases

From a uniform free stream, the flow behind the base starts at the free-stream speed: the base acts as a piston and empties the region behind it. The pressure and the temperature there fall towards zero before the wake fills. The pimple template has a temperature limit and survives this transient. `rhoCentralFoam` reads no `fvOptions` in v2412, has no such limit, and fails with a floating point exception in the thermophysical model. Therefore each supersonic chain starts with a pimple run on the coarsest mesh, and the central run starts from it (Figure 1.2).

Chapter 9

Results and measurement

Each run records its coefficients, its samples, its cost and the state of the computer in small files. A run thus can be compared with another solver without the fields of gigabytes.

9.1 Outputs of a run

Table 9.1: Outputs of a run. `<t0>` is the start time: 0 for a new run.

File	Contents
`postProcessing/forceCoeffs1/<t0>/coefficient.dat`	C_D, C_L, `CmPitch`, `CmRoll`, `CmYaw`, C_s, each also split into front `(f)` and rear `(r)`.
`postProcessing/forces1/<t0>/force.dat`, `moment.dat`	Force and moment: total, pressure and viscous parts.
`postProcessing/residuals/<t0>/solverInfo.dat`	Initial residuals.
`postProcessing/yPlus/<t0>/yPlus.dat`	y^+ minimum, maximum and mean for each wall patch, at each write time.
`postProcessing/timeInfo/<t0>/timeInfo.dat`	CPU and wall time, total and for each interval.
`postProcessing/surfaces/<t>/`	VTK of the skin and the symmetry plane, at the end.
`postProcessing/samples/<t>/*.csv`	Lines and points for the comparison with another solver, at the end (Table 9.2).
Last time directory	`p`, `T`, `U`, `rho`, `k`, `omega`, `nut`, `alphat`, `Ma`, `static(p)_coeff` (C_p), `yPlus`, `wallShearStress`.
`run.info`	Record of the case and of each execution (Section 9.2).
`log.<app>.<date>.xz`	Compressed logs. Read them with `xzless` or `xzgrep`.
`log.perf.<date>.xz`, `log.energy.<date>.xz`, `log.machine.<date>`	Measurement of the computer (Section 9.3).

Open the fields in ParaView with `paraview case.foam`. These quantities are not written, but `postProcess` calculates them from the last time without a new run:

Quantity	Command
Vorticity, Q criterion	`postProcess -func vorticity -latestTime`, `-func Q`
Total pressure and temperature	function object `pressure` with `mode total`; $T_0 = T\,(1 + \frac{\gamma-1}{2} M^2)$

Table 9.2: Samples for the comparison with another solver (function object `samples`).

Set	Contents
`skinZ0`	The wall cut by a plane slightly off $z = 0$. Columns x, y, z: windward is $y < 0$, leeward is $y > 0$.
`skinY0`	The wall cut by a plane slightly off $y = 0$ (the side). The two planes pass between the fins.
`wake05`, `wake1`, `wake2`, `wake5`	Lines across the wake on $z = 0$, at increasing distances behind the base, in body diameters.
`axis`	Along the body axis, from upstream of the nose to far into the wake.
`freeStream`	One point upstream of the nose tip.

The samples contain p, T, U, ρ, M, k, ω, ν_t, C_p, τ_w and y^+, with `cellPoint` interpolation.

9.2 run.info

`newCase.sh` makes `run.info`. `Allrun` adds a record `run_<date>` each time that it runs. The file is an OpenFOAM dictionary: read it with `foamDictionary`.

Table 9.3: Entries of `run.info`.

Entry	Written by	Contents
`case`, `created`, `commit`	`newCase.sh`	Name, date, `git describe --dirty` of the repository.
`mesh`, `nCells`, `options`, `init`	`newCase.sh`	Mesh, number of cells, options, start run.
`meshHash`, `flow`	`newCase.sh`	Fingerprint of the mesh and physical definition of the case (Section 10.1).
`host`, `cpu`, `np`, `binding`	`Allrun`	Computer, processes, binding (Section 6.2.3).
`start`, `end`, `wallSeconds`	`Allrun`	Wall time of decomposition, solver and reconstruction.
`solverWallSeconds`	`Allrun`	Wall time of the solver only.
`status`, `steps`, `simTime`	`Allrun`	`ok` if the solver log ends with `End`; steps; first and last time.
`maxRSSkB`	`Allrun`	Peak memory of the largest MPI process.
`coreUsPerCellStep`	`Allrun`	wall $\cdot\, n_p \cdot 10^6 / (N_{\text{cells}} \cdot \text{steps})$.

Entry	Written by	Contents
`cyclesPerCellStep`	`Allrun`	CPU cycles of all processes $/(N_{\text{cells}} \cdot \text{steps})$.
`Cd`	`Allrun`	Last value of `coefficient.dat`.
`machine`, `measured`	`Allrun`	State of the computer and measurement (Table 9.4).

9.3 Measurement of the computer

The wall time of a run depends on the computer as much as on the solver: CPU profile, turbo, temperature, suspension. `Allrun` measures the computer during the solver, without a change to the solver (Figure 9.1).

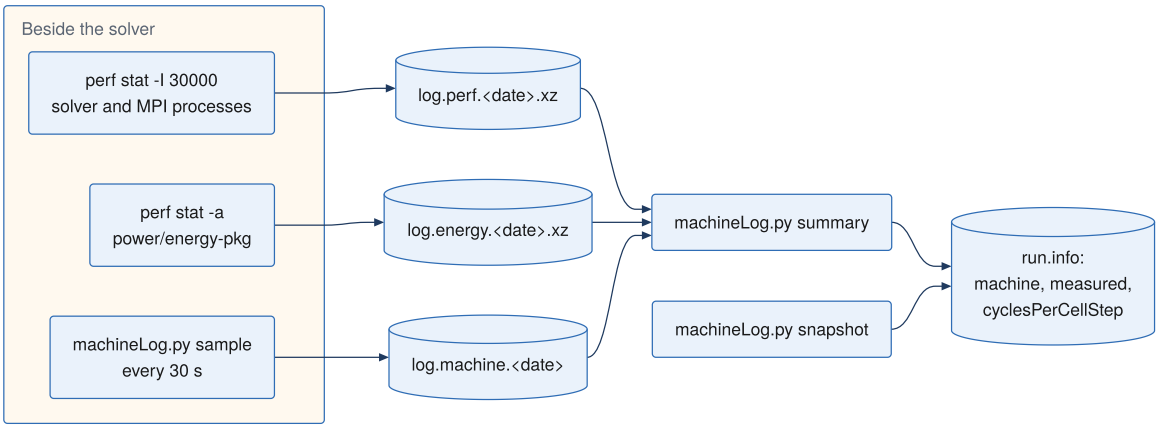


Figure 9.1: Measurement during the solver and its record in `run.info`.

File	Tool	Contents
`log.perf`	`perf stat -I` on the solver	At a fixed interval: task clock, cycles, instructions, front-end stalls, cache misses, migrations, context switches.
`log.machine`	`machineLog.py sample`	At a fixed interval: wall, monotonic and boot clocks, CPU frequency, load, temperatures of CPU, memory and NVMe, fans, CPU profile.
`log.energy`	`perf stat -a -e power/energy-pkg/`	Energy of the CPU package at a fixed interval. Only if `perf` can count on the whole system.

Table 9.4: Block `measured` of `run.info`.

Entry	Meaning
`suspendedSeconds`	Change of the boot clock minus change of the monotonic clock: the time that the computer was suspended. If it is not 0, the wall time is not a cost.

Entry	Meaning
`mhzMean`, `mhzMinOfMean`	Mean clock of all logical CPUs, and the lowest mean of one interval.
`tCpuMax`, `tDimm<i>Max`, `tNvmeMax`	Highest temperatures. Memory above its rated temperature refreshes more often and loses bandwidth.
`profiles`	The CPU profiles during the run. If it is not only `performance`, the run does not compare.
`cpuSeconds`	CPU time of all solver processes.
`cycles`, `instructions`	Totals of all processes.
`GHzEffective`	cycles/cpuSeconds: the real clock of the solver, with turbo and throttling.
`IPC`	instructions/cycles. Low ($\lesssim 1$) in a code that the memory bandwidth limits.
`frontendStallFraction`	Cycles with a stopped front end, divided by the cycles.
`cacheMisses`, `cpuMigrations`, `contextSwitches`	Indicators of noise. Many migrations show a binding that does not work.
`energyPkgJ`	Energy of the CPU package during the solver.

The block `machine` gives the state at the start: kernel, sockets, cores, threads for each core, memory, CPU governor and energy preference, boost, maximum frequency, CPU profile, fan level, `perf_event_paranoid`, OpenFOAM build and Open MPI version.

`cyclesPerCellStep` is the cost that does not depend on the clock. It includes the start (the solver reads the mesh), which is small in a run of thousands of steps.

9.3.1 Requirements

- `kernel.perf_event_paranoid` ≤ 1 to count the cycles. If not, `Allrun` shows a message with `!!` and runs without `cycles`.
- ≤ 0 (or `CAP_PERFMON`) to measure the energy (Section 3.4).

9.3.2 Procedure to get a comparable cost

1. Do the steps of Section 3.4.
2. Run one case at a time, with nothing heavy beside it.
3. Use the same number of processes and the same binding for the two solvers: one process for each physical core.
4. Before you use the cost of a run, examine `suspendedSeconds`, `profiles`, `GHzEffective` and `cpuMigrations`.

9.3.3 Do not divide by the clock frequency

A correction of the wall time by the clock supposes that the time changes as $1/f$. This is true for a code that the computation limits, not for a code that the memory bandwidth limits. The two solvers differ in this property: OpenFOAM (low order) depends more on the memory

bandwidth, Nektar++ at a high polynomial order more on the clock. To measure it, run the same short case at several clock limits (`scaling_max_freq` at 2, 3 and 4 GHz) and fit

$$t(f) = t_{\mathrm{mem}} + \frac{C}{f}.$$

This procedure is not automated.

9.3.4 Runs without this measurement

A run that has only `timeInfo.dat` gives CPU and wall time of one process at each interval. An interval with wall time much larger than CPU time is a suspension. An interval with CPU time much above the median is a low clock or a load beside the run. Leave out these intervals before you use the cost.

9.4 Monitor a run

```
./post/monitor.sh <run-dir> [seconds]
```

The script plots C_D and C_L against the iteration and the initial residuals on a logarithmic scale, and draws them again every `[seconds]`. It joins the directories of a continued run. Without a display, it writes `<run-dir>/monitor.png`. It needs `gnuplot`.

9.5 Table of a sweep

```
./post/sweepTable.sh [runs-dir] > sweep.csv    # default □/runs
```

One line for each run. A run that did not start has empty fields.

Columns	Source
`case`, `mesh`, `nCells`	`run.info`
`solver`	`application` in `controlDict`
`Minf`, `pInf`, `Tinf`, `alpha`	`flowConditions`
`status`, `steps`, `tEnd`, `wallSeconds`	Last record of `run.info`
`Cd`, `Cl`, `CmPitch`	Last line of `coefficient.dat`
`window`, `CdMean`, `ClMean`, `CmPitchMean`	Mean over the window of the stop criterion
`resUx` … `resOmega`	Last initial residuals. `resP` is empty for `rhoCentralFoam`.
`yPlusMin`, `yPlusMax`, `yPlusAvgWalls`, `yPlusAvgFins`	y^+ at the last time

With a continued run, the coefficients and residuals come from the last start-time directory only.

9.6 Disk

- The solver log is compressed with `xz` at the end. While the solver runs, the log grows by 1 to 2 kB for each step.

- The residuals and the coefficients write one line at each output interval.
- `constant/polyMesh` is a link to the mesh, except with `--refine`.
- `purgeWrite 1` keeps only the last time. `processor*` is removed when `reconstructPar` ends correctly.

CAUTION

Many cases link to one mesh. If you remove or make a mesh again, all the cases that use it lose their mesh.

9.7 Notification

`Allrun` sends a POST request at the end of each run, with the case name, `ok` or `failed`, the time, C_D and the hours of wall time. With ntfy, write the URL of a topic in a file:

```
echo https://ntfy.sh/<topic> > ~/.config/aconcagua/notify
```

The topic is public for each person who knows its name: use a name that is difficult to guess. The environment variable `NOTIFY_URL` has priority over the file. No notification comes if `runCase.sh` is stopped or if the computer stops.

Chapter 10

Comparison with Nektar++

The aim is to run all the Aconcagua cases with OpenFOAM and with Nektar++ and to compare the two solvers. Nektar++ adds p-refinement to the h-refinement of the meshes, so it has more cases.

"The same case" is the same mesh, the same free stream and the same initial field. The schemes, the time integration, the compressible formulation and the turbulence treatment can differ, but they must be as near as possible. This chapter gives what each OpenFOAM run records for the Nektar++ run, and what to measure in the same way in the two solvers.

10.1 What must be the same

Table 10.1: Items that the Nektar++ run must reproduce.

Item	Where it is	Note
Mesh	`meshHash` in `run.info`; `<mesh-dir>/meshHash`	Hash of `points`, `faces`, `owner`, `neighbour`, `boundary`. With `--refine`, the hash of the refined mesh. The Nektar++ mesh comes from the same `polyMesh`.
Half mesh	`meshInfo`: `sector half`	Symmetry plane $z = 0$. `Aref` is half the frontal area.
Free stream	block `flow` in `run.info`	`Minf`, `alpha`, `pInf`, `Tinf`, `Uinf`, `rhoInf`, `muInf`, the components of **U**.
Reynolds number	`RePerMetre`, `ReD`, `ReL`	From `rhoInf`, `Uinf` and `muInf`, on the diameter and on the body length.
Gas	`Rgas`, `gammaGas`, `Cp`, `RgasThermo`	`Rgas` of `flowDerived` and `RgasThermo` from `molWeight`: use the one of the thermophysical model (Section 8.7).
Viscosity	`sutherland (As Ts)`	$\mu = A_s\sqrt{T}/(1 + T_s/T)$.
Conductivity	`PrEucken`	Pr from the Eucken correction, not the usual 0.72 (Section 8.7). Nektar++ must use this value.
Turbulence	`turbulence`, `Prt`, `Ti`, `nuRatio`, `kInlet`, `omegaInlet`	k–ω SST with `nutUSpaldingWallFunction`.

Item	Where it is	Note
Wall	`wallT`, `wallNut`	Adiabatic (`zeroGradient` on T), no slip.
Initial field	`initial`	`uniform free stream`, or the run that was mapped.
References	`FOcommon`	`CofR (0 0 0)` (nose tip); drag and lift in wind axes; `lRef`; `Aref`.

10.2 What to compare

Quantity	OpenFOAM file
History of C_D, C_L, C_m	`forceCoeffs1/*/coefficient.dat`,
Pressure and viscous force	`forces1/*/force.dat`
C_p, τ_w, y^+ along the body	`samples/*/skinZ0_*.csv`, `skinY0_*.csv`
Wake profiles	`samples/*/wake05`, `wake1`, `wake2`, `wake5`
Axis and free-stream check	`samples/*/axis_*.csv`, `freeStream_*.csv`
Skin and symmetry plane	`surfaces/*/` (VTK)
Cost	`cyclesPerCellStep`, `cpuSeconds`, `energyPkgJ`, `GHzEffective` in `run.info`

10.3 How to measure in the same way

- **Degrees of freedom.** OpenFOAM: N_{cells} for each variable. Nektar++: $N_{\text{elements}}\,(P+1)^3$ for each variable on hexahedra (a different count on prisms and tetrahedra). Compare the cost for each degree of freedom and each step, in cycles.
- **Convergence.** Each solver stops with its own criterion and its own step. For the cost to the result, apply one criterion to the two histories of C_D, against the accumulated cost. Do not use the residuals: each code normalizes them differently.
- **Refinement.** Each pair (h, P) of Nektar++ is a case. The cost for each degree of freedom puts them on the same curve as $h1$ to $h3$ of OpenFOAM.
- **Computer.** Use the procedure of Section 9.3, the same computer, the same number of processes and the same binding. Do not divide by the clock frequency: measure the dependence on the clock.

Chapter 11

Flight data

`flightData/cdFlight.py` uses the telemetry of a flight computer to calculate the atmosphere of the day, the speed, the drag coefficient of the coast phase and the flight conditions of a study. It plots the flight C_D against the CFD runs and, optionally, an OpenRocket simulation.

11.1 Command

```
python3 flightData/cdFlight.py [--csv <telemetry>] [--mass <kg>] [--runs <runs-dir>]
        [--ork <file.ork>] [--alphas <list>] [--tanchor <s>] [--out <png>]
```

`python3 flightData/cdFlight.py -h` gives the defaults. The mass is the mass after burnout, read from the OpenRocket simulation in `flightData/Aconcagua.ork` or given with `--mass`: C_D is proportional to it.

Telemetry format. A CSV file with a comma between fields and a dot as decimal sign, without thousands separators, with the column names of the TeleMega export (AltOS): `time`, `state_name`, `pressure`, `speed`, `accel_x`, `tilt`, `pad_range`, `pad_dist`, `nsat`. The script stops on a decimal comma and on a value outside the physical limits of `LIMITS` in the script, with the line and the column. To use another flight computer, convert its export to these names and units first. The accelerometer saturates during the motor burn; the script uses only the coast.

11.2 Method

1. **Atmosphere.** The ground pressure p_0 is the mean before launch. During the descent the barometer sees no shocks, and the GPS gives the height above the pad from the slant range and the horizontal distance. A fit of

$$p = p_0 \left(1 - \frac{L\,h}{T_0}\right)^{g/(RL)}$$

   to these pairs gives the temperature at the pad T_0 and the lapse rate L of the day. Hydrostatic equilibrium is the only assumption.

2. **Speed.** The speed of the flight computer converts pressure to height with the standard atmosphere, and its barometer is behind shocks at high speed. The script integrates instead the axial accelerometer along the flight path,

$$\frac{dV}{dt} = a_x - g\cos\theta,$$

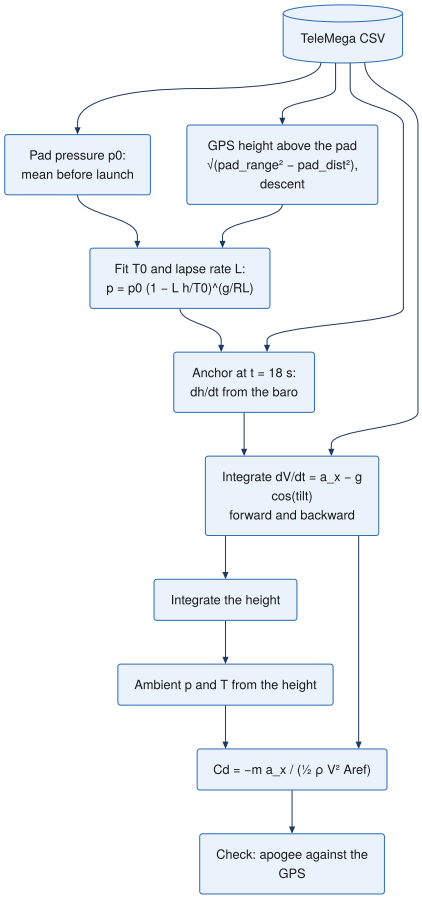


Figure 11.1: Calculation of the flight drag coefficient, from the telemetry (top) to the result (bottom).

with θ the tilt from the vertical, forward and backward from an anchor time in the subsonic coast, where the barometric height with the fitted atmosphere gives the speed.

3. **Height and ambient state.** The integral of $V \cos\theta$ gives the height; the fitted atmosphere gives p and T at that height.

4. **Drag.** With the motor off, the axial accelerometer measures the specific force $-D/m$:

$$C_D = \frac{-m\, a_x}{\frac{1}{2}\rho V^2 A_{\text{ref}}}.$$

   The data start when the residual thrust ends.

5. **Check.** Compare the integrated apogee with the GPS apogee.

6. **Flight conditions.** For each Mach number of the study, the script prints the time, the height, p_∞ and T_∞.

11.3 Angle of attack of the flight

A rolling rocket gives two lateral accelerations:

- a centripetal acceleration of a sensor off the axis, $\omega_{\text{roll}}^2 r$, fixed in the body axes;
- the aerodynamic normal force, fixed in space, which turns in the body axes at the roll frequency.

The mean of the lateral acceleration, divided by ω_{roll}^2, must give a constant radius r. The oscillation at the roll frequency gives the normal force N, so $C_N = N/(\frac{1}{2}\rho V^2 A_{\text{ref}})$ and $\alpha \approx C_N/C_{N\alpha}$, with $C_{N\alpha}$ from the CFD. The sensor noise is in the oscillation, so the result is an upper limit of α.

11.4 Sources of uncertainty

Source	Effect
Mass	$C_D \propto m$. A design mass is not a measurement.
Wind	The speed is relative to the ground.
Flight path	The gravity term uses the attitude as the direction of the path; valid at small α.
Atmosphere	One lapse rate for the whole column; real layers deviate from it.

Chapter 12

Validation

Do the procedures in this sequence: the checks on each run, the subsonic references, the transonic and supersonic references, the flight, and the mesh convergence before you publish a value.

12.1 Checks on each run

Table 12.1: Checks on each run.

Check	Correct result	Cause if not correct
C_L, C_m at $\alpha = 0$; C_l (roll)	≈ 0	The mesh is not symmetric, or the run did not converge.
y^+ on each wall patch	Near the estimate of Section 5.3, and equal on the meshes of a study	The first cell changed.
Minimum and maximum of T	Physical: at most near the stagnation temperature	The start transient is not resolved.
Iterations	More than `minIter`, and the criterion met	The run stopped before it was steady.
$C_N(\alpha)$	Linear at small α	The run did not converge, or the fins are not resolved.
Machine record	No suspension; one CPU profile	The cost is not usable (Section 9.3).

12.2 Subsonic flow: Barrowman

The slender-body equations of Barrowman [20], Barrowman and Barrowman [21], which OpenRocket uses, give the normal force and the centre of pressure of each component, referred to the body cross-section and for each radian. Fleeman [22, ch. 2] gives equivalent equations for the fins and the fin-body interference.

Component	$C_{N\alpha}$
Nose	2, for any nose of revolution
Cylinder	≈ 0
Boattail	$2\left[(d_{\text{base}}/d)^2 - 1\right]$
Fins, N fins	$K_{fb}\dfrac{4N(s/d)^2}{1+\sqrt{1+\left(\dfrac{2l_m}{c_r+c_t}\right)^2}}, \quad K_{fb} = 1 + \dfrac{r}{s+r}$

s is the exposed semi-span, c_r and c_t the root and tip chords, l_m the length of the mid-chord line, r the body radius.

1. Run an angle-of-attack sweep at small angles.
2. Calculate the slope of $C_N = C_L \cos\alpha + C_D \sin\alpha$ against α in radians. C_L is normal to the free stream, C_N to the body axis.
3. Compare the slope with the sum of the components. For the breakdown, add one `forceCoeffs` function object for each patch.

Drag at zero angle. Compare the components, not only the total. The viscous part of `force.dat` against the skin friction $C_f S_{\text{wet}}/A_{\text{ref}}$ with $C_f = 0.0576\, Re_L^{-1/5}$ [23, ch. 21] and the form factors $1 + 1/(2L/d)$ (body) and $1 + 2t/c$ (fins); the base drag against Hoerner [24, ch. 3]. Re_L of each case is in `run.info`.

12.3 Transonic and supersonic flow

1. **Tutorial of the template.** Run the `rhoCentralFoam` tutorial `biconic25-55Run35` and compare it with its published data. This separates a configuration error from a mesh error.
2. **Oblique shock on a wedge.** The θ–β–M relation is exact [5, ch. 4]:

$$\tan\theta = 2\cot\beta\,\frac{M^2\sin^2\beta - 1}{M^2(\gamma + \cos 2\beta) + 2}.$$

   The tutorial `wedge15Ma5` is this case.
3. **Flight.** Chapter 11. The CFD must show the transonic drag rise of the flight. If it does not, the mesh is too coarse where the shocks occur.
4. **OpenRocket.** Semi-empirical: it finds gross errors, not small ones.

12.4 Mesh convergence

With f_1, f_2, f_3 the results on the fine, medium and coarse meshes, the representative cell size $h = N^{-1/3}$ and the ratio $r = h_{\text{coarse}}/h_{\text{fine}}$, the method of Celik et al. [9] gives the observed order p, the extrapolated value and the grid convergence index:

$$p = \frac{\ln\left[(f_3 - f_2)/(f_2 - f_1)\right]}{\ln r}, \qquad f_{\text{ext}} = f_1 + \frac{f_1 - f_2}{r^p - 1}, \qquad \text{GCI}_{12} = 1.25\,\frac{|(f_1 - f_2)/f_1|}{r^p - 1}.$$

With unequal ratios, use the iterative form of Celik et al. [9]. If $(f_3 - f_2)/(f_2 - f_1) < 0$, the convergence is oscillatory and the equations do not apply [25].

12.5 Record

For each published value keep: `meshDict`, `meshInfo` and `log.checkMesh` of the mesh; the STL file or its commit; `run.info`, `flowConditions` and `config.json` of the run; and `postProcessing/`. With these files another person can make the result again.

Chapter 13

Troubleshooting

Table 13.1: Symptoms, causes and remedies.

Symptom	Cause	Remedy
`source the OpenFOAM bas hrc first`	No OpenFOAM environment.	Chapter 3.
Negative cell volumes.	The surface breaks a rule of Table 4.2, or a level is too coarse for a curvature.	Do not use the mesh. Correct the surface or the levels.
A wall patch without layers.	No entry in `patchBoundaryLayers`.	Section 5.3.
Incorrect domain, or a number with a decimal comma.	`awk` in a locale with a decimal comma.	Section 13.3.
`rhoCentralFoam` fails in the first iterations.	Start from a uniform free stream.	Section 8.12.
Courant number of order 10^7 in the log.	Normal with `localEuler`.	Section 13.2.
A run with `--init` behaves as a start from rest.	`mapFields` failed.	Section 13.1.
`taskset` shows all CPUs for a process; many migrations.	The binding does not work.	Section 6.2.3.
Wall time much larger than CPU time; `suspendedSeconds` > 0.	The computer suspended.	Section 3.4. The coefficients are correct; the cost is not.
`profiles` has more than one value; low `GHzEffective`.	The CPU profile changed during the run.	Section 3.4.
No `cycles` in `run.info`.	`perf_event_paranoid` too high.	Section 3.4.
The computer swaps or the run stops for memory.	The mesh is too large for the memory.	Scale the memory from a coarser mesh (Table 3.1).
`pkill -f` stops your own shell.	The command line contains the pattern.	Stop the process by its number.

Symptom	Cause	Remedy
A sweep reads wrong lines.	The sweep file was edited while it ran.	Stop, edit, start again.
A script does unexpected things after an edit.	`sh` reads a script while it runs it.	Do not edit a script that runs.
A dictionary does not parse near its banner.	`*/` in the banner comment closes the comment.	Remove `*/`.
A script continues after a failed application.	`runApplication` returns 0 on `FOAM FATAL ERROR`.	Read each log, as `Allmesh` and `Allrun` do.

13.1 mapFields on the same mesh

`mapFields` can stop with a segmentation fault when the source and the target are the same mesh. Between different meshes it works. `Allrun` stops the run if the log of `mapFields` or `changeDictionary` does not end with `End`; a check for `FOAM FATAL` only does not detect a segmentation fault. On the same mesh, a copy of the fields of the source run is an alternative that does not interpolate.

13.2 The Courant number with a local time step

With `localEuler`, `rhoCentralFoam` calculates the Courant number with the global `deltaT`, so the value in the log is meaningless. Read `Flow time scale min/max` (Section 8.5.1).

13.3 Decimal comma

In a locale with a decimal comma, `awk` reads and writes numbers with a comma, and OpenFOAM reads them incorrectly. Each script that calculates with `awk` sets `LC_ALL=C`. Do the same in a new script.

13.4 Rules for a change to the scripts

- Measure before you propose a change; a change without a number is not complete.
- Do not put a Python script in a bash here-document: a `\n` in an f-string becomes a real new line and breaks the file.
- After a change, run one short case and examine `run.info`.
- Change this guide in the same commit.

Appendix A

Quick reference

A.1 Commands

```
source /usr/lib/openfoam/openfoam2412/etc/bashrc

# geometry and one mesh
python3 mesh/makeRocket.py mesh/stl/<name>.stl --finRoll <deg>
mesh/Allmesh □/meshes/<mesh> [--maxCellSize <m>] [--nLayers <n>] [--thicknessRatio <
    r>]
./post/firstLayer.py □/meshes/<mesh>

# one case
./newCase.sh □/runs/<case> --mesh □/meshes/<mesh> --solver pimple --Minf <M> --np <N
    >
cd □/runs/<case> && ./Allrun

# study
./runMesh.sh meshes.txt □/meshes
nohup ./runCase.sh <sweep-file> > □/log.<sweep> 2>&1 &

# results
./post/monitor.sh □/runs/<case>
./post/sweepTable.sh □/runs > sweep.csv
python3 flightData/cdFlight.py

# binding of a running case
for p in $(pgrep -x rhoPimpleFoam); do taskset -cp $p; done
```

A.2 Run names

`m<M>-a<alpha>-h<n>[-pimple]`: Mach number times ten, angle of attack in degrees, mesh. `-pimple` marks the `rhoPimpleFoam` start run of a supersonic chain.

A.3 Where each value is

Value	File
Refinement levels, layers, `maxCellSize`	`mesh/system/meshDict`; per mesh in `meshes.txt`
Domain distances	`mesh/Allmesh`
Reference area and length, body and fin data	`mesh/meshInfo`
Flight condition	`system/flowConditions`; per case in the sweep file
Courant ramp, stop criterion	`config.json` of each template
Gas and turbulence constants	`constant/` and `system/flowDerived` of `case-central/`
Output intervals, samples	`system/FOs/FOcommon`, `controlDictBase.j2`

Appendix B

Glossary

B.1 Terms and abbreviations

Term	Definition
Binding	The assignment of each MPI process to fixed CPU cores.
Boattail	The cone at the rear of the body, from the body diameter to the base.
Chain	A sequence of runs at one flight condition, each started from the run on the coarser mesh.
Coast	The flight phase after the motor burnout.
CFD	Computational fluid dynamics.
DOF	Degree of freedom: one unknown of the discrete system.
GCI	Grid convergence index [9].
IPC	Instructions per cycle.
ISA	International Standard Atmosphere.
LTS	Local time stepping (`localEuler`).
Mesh $h_0, h_1, \ldots$	The meshes of a study, from coarse to fine.
MSL	Mean sea level.
RANS	Reynolds-averaged Navier–Stokes equations.
Run	One case directory and its executions.
SST	Shear-stress transport (k–ω SST model).
Sweep	A file with one run on each line, and its execution by `runCase.sh`.
Template	`case-central/` or `case-pimple/`: the files from which `newCase.sh` makes a case.
TVD	Total variation diminishing.
y^+	Distance of the first cell centre from the wall, in wall units: $y\,u_\tau/\nu$.

B.2 Symbols

Symbol	Definition	Unit
A_{ref}	Reference area	m^2
a	Speed of sound	m/s
C_D, C_L, C_N	Drag, lift and normal-force coefficients	–
C_m	Pitching-moment coefficient about the nose tip	–
C_p	Pressure coefficient	–
c_p, c_v	Specific heats	J/(kg K)
d	Body diameter	m
k, ω	Turbulent kinetic energy, specific dissipation rate	m^2/s^2, $1/\text{s}$
M	Mach number	–
p, T, ρ	Pressure, temperature, density	Pa, K, kg/m^3
Pr, Pr_t	Laminar and turbulent Prandtl numbers	–
R	Gas constant of air	J/(kg K)
Re	Reynolds number	–
t_1	Thickness of the first wall cell	m
α, β	Angle of attack, angle of sideslip	deg
γ	Ratio of specific heats	–
μ, ν, ν_t	Dynamic, kinematic and eddy viscosity	Pa s, m^2/s
τ_w	Wall shear stress	Pa

Bibliography

[1] ISO/IEC/IEEE. ISO/IEC/IEEE 26514:2022 systems and software engineering – design and development of information for users, 2022.

[2] ASD Simplified Technical English Maintenance Group. ASD-STE100 simplified technical english: International specification for the preparation of technical documentation in a controlled language, 2021. Issue 8.

[3] H. K. Versteeg and W. Malalasekera. *An Introduction to Computational Fluid Dynamics: The Finite Volume Method*. Pearson, 2nd edition, 2007.

[4] F. Moukalled, L. Mangani, and M. Darwish. *The Finite Volume Method in Computational Fluid Dynamics: An Advanced Introduction with OpenFOAM and Matlab*. Springer, 2016.

[5] J. D. Anderson. *Modern Compressible Flow*. McGraw-Hill, 3rd edition, 2003.

[6] D. C. Wilcox. *Turbulence Modeling for CFD*. DCW Industries, 3rd edition, 2006.

[7] S. B. Pope. *Turbulent Flows*. Cambridge University Press, 2000.

[8] J. Martí Valverde and C. Stemmer. OpenFOAM ToolChain for the aerodynamics of sounding rockets. Technical report, Zenodo, 2026.

[9] I. B. Celik, U. Ghia, P. J. Roache, C. J. Freitas, H. Coleman, and P. E. Raad. Procedure for estimation and reporting of uncertainty due to discretization in CFD applications. *Journal of Fluids Engineering*, 130:078001, 2008.

[10] A. Kurganov and E. Tadmor. New high-resolution central schemes for nonlinear conservation laws and convection-diffusion equations. *Journal of Computational Physics*, 160: 241–282, 2000.

[11] A. Kurganov, S. Noelle, and G. Petrova. Semidiscrete central-upwind schemes for hyperbolic conservation laws and Hamilton–Jacobi equations. *SIAM Journal on Scientific Computing*, 23(3):707–740, 2001.

[12] C. J. Greenshields, H. G. Weller, L. Gasparini, and J. M. Reese. Implicit and explicit schemes for flows of aerodynamic and turbomachinery fluids. *International Journal for Numerical Methods in Fluids*, 63:1–21, 2010.

[13] B. van Leer. Towards the ultimate conservative difference scheme. V. A second-order sequel to Godunov's method. *Journal of Computational Physics*, 32:101–136, 1979.

[14] F. R. Menter. Two-equation eddy-viscosity turbulence models for engineering applications. *AIAA Journal*, 32(8):1598–1605, 1994.

[15] F. R. Menter, M. Kuntz, and R. Langtry. Ten years of industrial experience with the SST turbulence model. In *Turbulence, Heat and Mass Transfer 4*, pages 625–632, 2003.

[16] D. B. Spalding. A single formula for the law of the wall. *Journal of Applied Mechanics*, 28 (3):455–458, 1961.

[17] F. M. White. *Viscous Fluid Flow*. McGraw-Hill, 3rd edition, 2006.

[18] W. M. Kays. Turbulent Prandtl number – where are we? *Journal of Heat Transfer*, 116: 284–295, 1994.

[19] T. J. Poinsot and S. K. Lele. Boundary conditions for direct simulations of compressible viscous flows. *Journal of Computational Physics*, 101:104–129, 1992.

[20] J. S. Barrowman. The practical calculation of the aerodynamic characteristics of slender finned vehicles. Master's thesis, Catholic University of America, 1967.

[21] J. S. Barrowman and J. A. Barrowman. A method for calculating the static margin of a slender missile, 1966.

[22] E. L. Fleeman. *Tactical Missile Design*. AIAA, 2nd edition, 2006.

[23] H. Schlichting. *Boundary-Layer Theory*. McGraw-Hill, 7th edition, 1979.

[24] S. F. Hoerner. *Fluid-Dynamic Drag*. Hoerner Fluid Dynamics, 1965.

[25] P. J. Roache. Perspective: a method for uniform reporting of grid refinement studies. *Journal of Fluids Engineering*, 116:405–413, 1994.